Práctica 3: Java RMI

Desarrollo de Sistemas Distribuidos. Grupo DSD-3.

Grado en Ingeniería Informática

Curso 2025-2026.

ETSIIT. Universidad de Granada.





UNIVERSIDAD
DE GRANADA

Autor:
Ángel Rodríguez Faya

Profesor:
José Ángel Segura Muros

Fecha de entrega:
24 de abril de 2026

Índice

1. Introducción **3**

2. Parte 1. Implementación de Ejemplos. **3**
2.1. Ejemplo 1. Cliente-Servidor básico. 3
2.1.1. `Ejemplo_I.java` (La Interfaz Remota) 4
2.1.2. `Ejemplo.java` (El Servidor) 4
2.1.3. `Cliente_Ejemplo.java` (El Cliente) 6
2.1.4. `launch.sh` (Script de Ejecución) 6
2.1.5. Diagrama de clases del ejemplo 1. 9
2.1.6. Ejemplo de compilación y ejecución del ejemplo 1. . . . 9
2.2. Ejemplo 2. Multihebra. 11
2.2.1. `Ejemplo_I.java` (La Interfaz Remota) 11
2.2.2. `Ejemplo.java` (El Servidor) 11
2.2.3. `Cliente_Ejemplo_Multi_Threaded.java` (El Cliente) . 13
2.2.4. `launch.sh` (Script de Ejecución) 14
2.2.5. Diagrama de clases del ejemplo 2. 15
2.2.6. Ejemplo de compilación y ejecución del ejemplo 2. Respondiendo a las preguntas 16
2.3. Ejemplo 3. Contador. 19
2.3.1. `icontador.java` (La Interfaz Remota) 19
2.3.2. `contador.java` (El Objeto Remoto) 20
2.3.3. `servidor.java` (El Servidor) 20
2.3.4. `cliente.java` (El Cliente) 21
2.3.5. `launch.sh` (Script para Ejecución) 22
2.3.6. Diagramas en UML del ejemplo 3. 23
2.3.7. Ejemplo de compilación y ejecución del ejemplo 3. . . . 25

3. Parte 2. Servidor Replicado. **26**
3.1. Introducción y Objetivos. 26
3.2. Arquitectura del Sistema. 27
3.2.1. `iDonacion.java` (La Interfaz Remota) 27
3.2.2. `Donacion.java` (La Lógica de Negocio) 30
3.2.3. `Servidor.java` (El Servidor) 40
3.2.4. `Cliente.java` (El Cliente) 42
3.2.5. `launch.sh` (Script para Ejecución) 46
3.3. Diagramas en UML. 48
3.3.1. Diagrama de clases. 48
3.3.2. Diagrama de clases. 49
3.4. Guía de Uso. Ejemplos de ejecución 50

3.4.1. Registrar donantes. 51
3.4.2. Realizar donaciones 52
3.4.3. Consultar total de donaciones (en todo el sistema) . . . 54
3.4.4. Consultar el historial de donaciones 55
3.4.5. Consultar el mayor donante. 55
3.4.6. Darse de baja. 56
3.4.7. Salir de la aplicación. 56

1. Introducción

En esta tercera práctica vamos a trabajar y aprender el manejo de los mecanismos para implementar programas distribuidos en Java, haciendo uso de su API **RMI** (***R**emote **M**ethod **I**nvocation*), que nos permite construir objetos distribuidos. Como entorno de funcionamiento se utilizará la máquina virtual de java (*JVM*, Java Virtual Machine) que sigue un enfoque *"Write Once, Run Anywhere"*, que su traducción viene a ser: "se codifica una vez y se ejecuta en cualquier sitio"), usando este enfoque para que se ejecute en ó por todas partes (*"run everywhere"*).

Los sistemas RMI se caracterizan porque existe un objeto delegado (que llamaremos *stub)*) que es el encargado de gestionar las invocaciones al objeto remoto, gestionando la serialización y deserialización de los parámetros y valores de retorno.

Las aplicaciones en RMI (denominadas **aplicaciones de objetos distribuidos**)están formadas por dos aplicaciones: el cliente y el servidor. El servidor es el encargado de crear los objetos remotos, hacer accesibles las referencias a dichos objetos remotos y esperar la invocación de métodos sobre dichos objetos remotos por parte de los clientes. Estos, los clientes, obtienen una referencia remota a uno o varios objetos remotos en el servidor e invocan métodos sobre los objetos remotos. RMI proporciona los mecanismos que permiten la comunicació y el intercambio de información entre el cliente y el servidor.

A continuación, veremos los tres ejemplos que nos dan en el guion de este práctica, los ejecutaremos y analizaremos que está sucediendo, así como mostrar sus archivos, pues considero que es más cómodo a la hora de corregir, tenerlo todo en un documento (hablo de tener el código fuente de los ejercicios más ejemplos de ejecución), que tener la necesidad de abrir el documento y el editor de código a la vez.

2. Parte 1. Implementación de Ejemplos.

2.1. Ejemplo 1. Cliente-Servidor básico.

Este ejemplo es una muestra de cómo se realiza una implementación básica de una aplicación Cliente-Servidor en Java RMI. Tenemos un servidor llamado `Ejemplo` que implementa un método `escribir_mensaje(int idProceso)`, que recibe un entero que representa el identificador del proceso (Cliente) que invoca a este método del servidor, y muestra por pantalla dicho identificador. Si se trata del proceso/hebra 0, entonces la duerme 5 segundos

y después continua como el resto de procesos.

Veamos cada uno de los archivos que forman este ejemplo. Es importante señalar que **este código ha sido modificado durante las sesiones de prácticas**, pues se trata de una práctica antigua y hay paquetes/componentes que actualmente están en estado obsoletos.

2.1.1. `Ejemplo_I.java` (La Interfaz Remota)

Es la interfaz que define qué métodos del servidor pueden ser llamados por el cliente a través de la red.

Su código es:

```
1  // Ejemplo_I.java
2  // Define la Interfaz remota
3
4  import java.rmi.Remote;
5  import java.rmi.RemoteException;
6
7  public interface Ejemplo_I extends Remote {
8      public void escribir_mensaje (int id_proceso) throws RemoteException;
9  }
```

2.1.2. `Ejemplo.java` (El Servidor)

Implementa la interfaz anteriormente definida, es decir, el método `escribir_mensaje`. Esta aplicación está a la espera de recibir peticiones. Aquí se han **realizado modificaciones**, ya que el `SecurityManager` del ejemplo está obsoleto (*deprecated*) a partir de Java 17, por lo que se han añadido las clases `SslRMIClientSocketFactory` y `SslRMIServerSocketFactory` para que la comunicación vaya cifrada a través de **SSL/TSL**.

Su código es:

```
1  // Ejemplo.java
2  // Implementa la Interfaz remota
3
4  import java.rmi.registry.LocateRegistry;
5  import java.rmi.registry.Registry;
6  import java.rmi.server.UnicastRemoteObject;
7  import javax.rmi.ssl.SslRMIClientSocketFactory;
8  import javax.rmi.ssl.SslRMIServerSocketFactory;
9
10 public class Ejemplo implements Ejemplo_I {
11     private static final int RMI_PORT = 2099;
12
```

```
13      public Ejemplo() {
14          super();
15      }
16
17      public void escribir_mensaje (int id_proceso) {
18          System.out.println("Recibida peticion de proceso: "+id_proceso);
19          if (id_proceso == 0) {
20              try{
21                  System.out.println("Empezamos a dormir");
22                  Thread.sleep(5000);
23                  System.out.println("Terminamos de dormir");
24              }
25              catch (Exception e) {
26                  System.err.println("Ejemplo exception:");
27                  e.printStackTrace();
28              }
29          }
30          System.out.println("\nHebra "+id_proceso);
31      }
32
33      public static void main(String[] args) {
34          try {
35              // Definimos el nombre del objeto remoto que se registrará en el ligador
36              String nombre_objeto_remoto = "Ejemplo_I";
37
38              // Creamos una instancia del objeto remoto
39              Ejemplo_I prueba = new Ejemplo();
40
41              // Creamos los socket factories para SSL
42              SslRMIClientSocketFactory csf = new SslRMIClientSocketFactory();
43              SslRMIServerSocketFactory ssf = new SslRMIServerSocketFactory();
44
45              // Creamos el stub del objeto remoto para el cliente, exportandolo al
46              // puerto 0 (puerto anonimo) y usando los socket factories creados.
47              Ejemplo_I stub =
48                  (Ejemplo_I) UnicastRemoteObject.exportObject(prueba, 0, csf, ssf);
49
50              // El Registry vive como proceso externo (rmiregistry).
51              Registry registry = LocateRegistry.createRegistry(RMI_PORT);
52
53              // Registramos el objeto remoto en el ligador
54              registry.rebind(nombre_objeto_remoto, stub);
55
56              // A partir de aquí, el objeto remoto ya puede ser invocado por los clientes.
57
58              System.out.println("Ejemplo bound");
59          }
60          catch (Exception e) {
61              System.err.println("Ejemplo exception:");
```

```
            e.printStackTrace();
        }
    }
}
```

2.1.3. `Cliente_Ejemplo.java` (El Cliente)

Es el encargado de conectar con el servidor para realizar peticiones. Para ello, se conecta al registro (el `rmiregistry`) que hace de guía para poder encontrar nuestros objetos, y busca el servidor por su nombre, que sería ".Ejemplo_I". Después se llama al método para escribir el mensaje, pasándole su identificador de proceso como parámetro.

Su código sería el siguiente:

```
// Cliente_Ejemplo.java
// Código del cliente que invoca al objeto remoto implementado en Ejemplo.java

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Cliente_Ejemplo {
    private static final int RMI_PORT = 2099;

    public static void main(String args[]) {
        try {
            String nombre_objeto_remoto = "Ejemplo_I";
            System.out.println("Buscando el objeto remoto");
            Registry registry = LocateRegistry.getRegistry(args[0], RMI_PORT);
            // Invocamos el objeto remoto obtenido del ligador
            Ejemplo_I instancia_local = (Ejemplo_I) registry.lookup(nombre_objeto_remoto);
            System.out.println("Invocando el objeto remoto");
            // Llamamos al método remoto, pasándole
            // el id del proceso (recibido como argumento)
            instancia_local.escribir_mensaje(Integer.parseInt(args[1]));
        } catch (Exception e) {
            System.err.println("Ejemplo_I exception:");
            e.printStackTrace();
        }
    }
}
```

2.1.4. `launch.sh` (Script de Ejecución)

Nos han proporcionado este script para poder realizar la compilación/ejecución con tan sólo ejecutarlo. Básicamente, lo que hace es crear

unos “certificados digitales” necesarios para que el SSL funcione en neustra máquina, compilar todos los .java de golpe, lanzar el servidor en segundo plano y por último lanzar dos clientes para que se conecten al servidor, estos clientes tienen como identificadores 0 y 3, respectivamente.

```
#!/bin/sh -e
# ejecutar = Macro para compilación y ejecución del programa ejemplo
# en una sola maquina Unix de nombre localhost.

KEYSTORE=server-keystore.p12
TRUSTSTORE=client-truststore.p12
STOREPASS=changeit
RMI_PORT=2099

if [ ! -f "$KEYSTORE" ]; then
        echo
        echo "Generando keystore TLS de ejemplo ..."
        keytool -genkeypair \
                -alias rmi-server \
                -keyalg RSA \
                -keysize 2048 \
                -validity 3650 \
                -dname "CN=localhost, OU=RMI, O=DSD, L=Local, ST=Local, C=ES" \
                -storetype PKCS12 \
                -keystore "$KEYSTORE" \
                -storepass "$STOREPASS" \
                -keypass "$STOREPASS" \
                -noprompt
fi

if [ ! -f "$TRUSTSTORE" ]; then
        echo
        echo "Generando truststore TLS de ejemplo ..."
        keytool -exportcert \
                -alias rmi-server \
                -keystore "$KEYSTORE" \
                -storetype PKCS12 \
                -storepass "$STOREPASS" \
                -rfc \
                -file rmi-server.crt

        keytool -importcert \
                -alias rmi-server \
                -file rmi-server.crt \
                -storetype PKCS12 \
                -keystore "$TRUSTSTORE" \
                -storepass "$STOREPASS" \
                -noprompt
```

```
44
45          rm -f rmi-server.crt
46  fi
47
48  # Si quedo un proceso previo en el puerto, lo cerramos para evitar conflictos.
49  if command -v fuser >/dev/null 2>&1; then
50          fuser -k "${RMI_PORT}/tcp" >/dev/null 2>&1 || true
51  fi
52
53  echo
54  echo "Compilando con javac ..."
55  javac *.java
56
57  sleep 2
58
59  echo
60  echo "Lanzando el servidor"
61  java -cp . \
62          -Djava.rmi.server.hostname=localhost \
63          -Djavax.net.ssl.keyStore="$KEYSTORE" \
64          -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
65          -Djavax.net.ssl.keyStoreType=PKCS12 \
66          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
67          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
68          -Djavax.net.ssl.trustStoreType=PKCS12 \
69          Ejemplo &
70
71  sleep 2
72
73  echo
74  echo "Lanzando el primer cliente"
75  echo
76  java -cp . \
77          -Djavax.net.ssl.keyStore="$KEYSTORE" \
78          -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
79          -Djavax.net.ssl.keyStoreType=PKCS12 \
80          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
81          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
82          -Djavax.net.ssl.trustStoreType=PKCS12 \
83          Cliente_Ejemplo localhost 0
84
85  sleep 2
86
87  echo
88  echo "Lanzando el segundo cliente"
89  echo
90  java -cp . \
91          -Djavax.net.ssl.keyStore="$KEYSTORE" \
92          -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
```

```
93          -Djavax.net.ssl.keyStoreType=PKCS12 \
94          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
95          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
96          -Djavax.net.ssl.trustStoreType=PKCS12 \
97          Cliente_Ejemplo localhost 3
```

2.1.5. Diagrama de clases del ejemplo 1.

El diagrama de clases en UML del ejemplo 1 es:

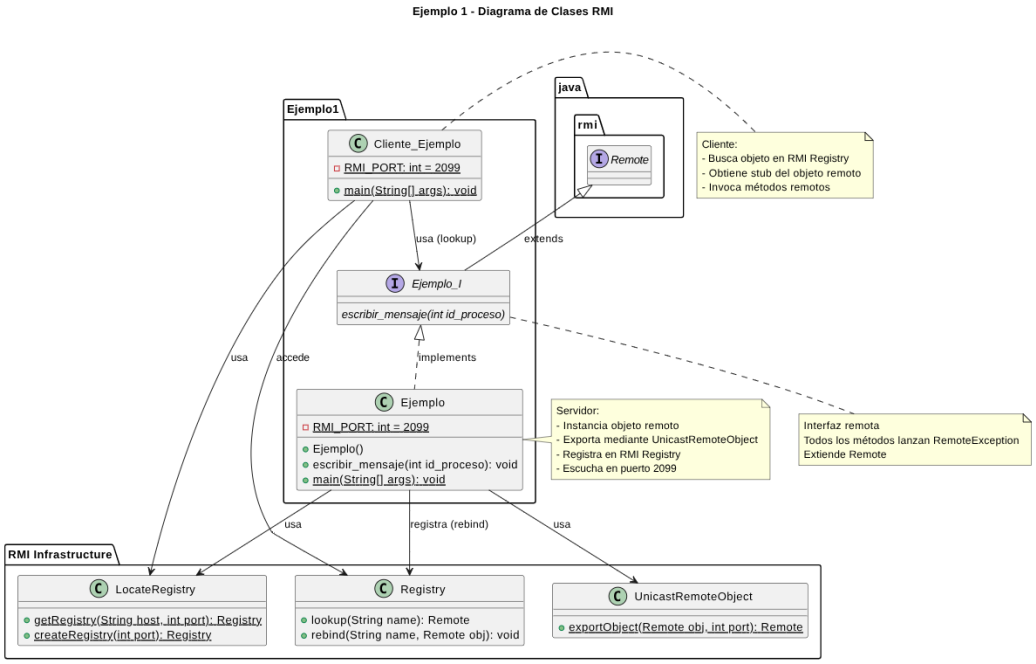


Figura 1: Diagrama de clases del ejemplo 1

2.1.6. Ejemplo de compilación y ejecución del ejemplo 1.

En primer lugar, debemos dar permiso de ejecución al script. Para ello abrimos una terminal y ejecutamos el siguiente comando:

```
$ chmod u+x launch.sh
```

Después, ejecutamos el script con el siguiente comando:

```
$ ./launch.sh
```

A continuación, muestro una imagen con la salida de la ejecución:

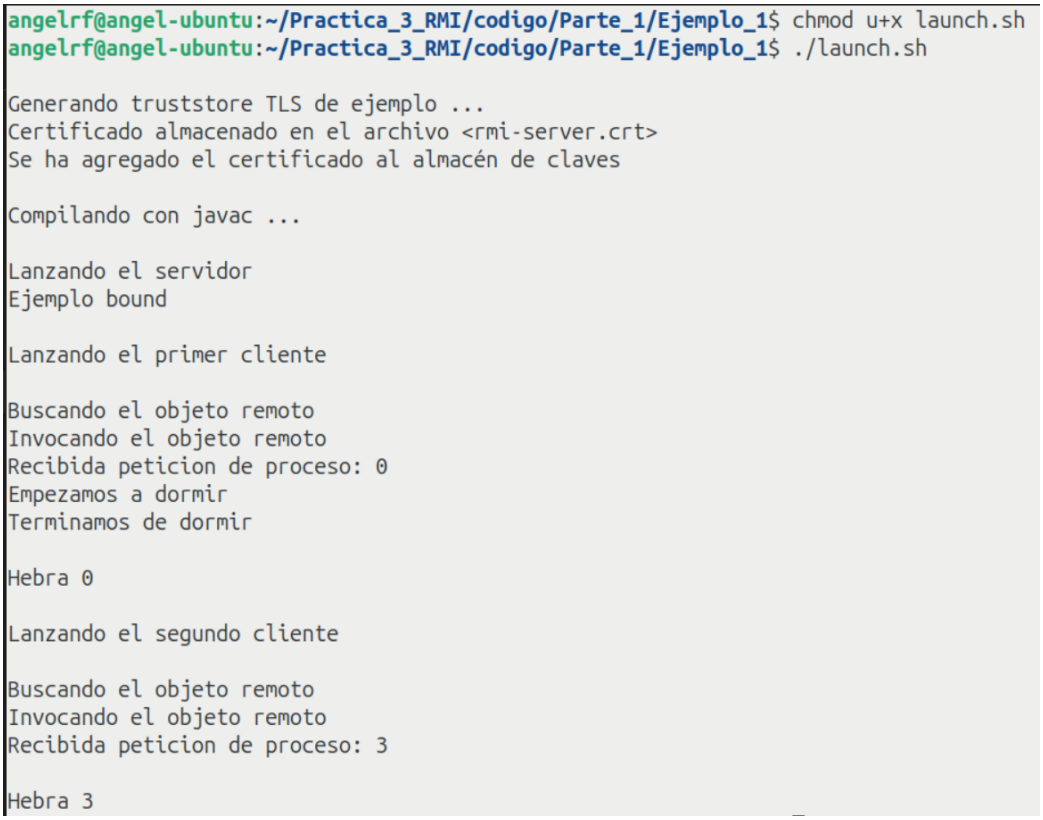


Figura 2: Ejecución del ejemplo 1

Como podemos observar en la salida, se compilan todos los archivos, se lanza el servidor y se muestra el mensaje `.Ejemplo bound"`, que nos indica que el objeto remoto ya puede ser invocado por los clientes. Después se lanza el cliente 0. Este busca el objeto remoto y lo invoca. El método de escribir mensaje en el servidor recibe la petición mostrando ese mensaje con el id del proceso cliente. Como se trata del proceso 0, lo pone a dormir 5 segundos. Pasado ese tiempo, muestra el mensaje de que ha terminado de dormir e imprime rl mensaje de hebra con el id de nuevo, y ya termina. Finalmente se lanza el cliente con id 3 (aunque no se lanza después de que termine la hebra 0, si no que **se lanza la hebra 3 mientras la hebra 0 duerme**).De nuevo, este cliente busca el objeto remoto y lo invoca. Llama al método para escribir mensaje, este muestra que le ha llegado la petición mostrando su id.

2.2. Ejemplo 2. Multihebra.

Partiendo del ejemplo anterior, vamos a realizar una modificación y es que no vamos a lanzar varios clientes como habíamos hecho en el ejemplo anterior, si no que vamos a crear múltiples hebras dentro de un mismo cliente. Estas hebras van a realizar la misma tarea, que será imprimir un mensaje remoto accediendo al *stub* de un objeto remoto. El objetivo de este ejemplo es dejar constancia de la gestión de concurrencia de RMI.

De nuevo, vamos a ver los archivos y comentar los cambios que se han realizado con respecto a la versión del ejemplo 1.

2.2.1. `Ejemplo_I.java` (La Interfaz Remota)

Se ha modificado el parámetro que recibe el método imprimir mensaje, que ahora es un `String` en lugar de un entero.

Su código es:

```
1  // Ejemplo_I.java
2  // Define la Interfaz remota
3
4  import java.rmi.Remote;
5  import java.rmi.RemoteException;
6
7  public interface Ejemplo_I extends Remote {
8      public void escribir_mensaje (String mensaje) throws RemoteException;
9  }
```

2.2.2. `Ejemplo.java` (El Servidor)

Se cambia la implementación del método `escribir_mensaje`. En este caso, se informa de cuando entra y cuando sale la hebra, cuyo nombre es el mensaje pasado como parámetro al método. Además, **las hebras que sean múltiplo de 10 (su nombre termina en 0)**, se dormirán durante 5 segundos

Su código es:

```
1  // Ejemplo.java
2  // Implementa la Interfaz remota
3
4  import java.rmi.registry.LocateRegistry;
5  import java.rmi.registry.Registry;
6  import java.rmi.server.UnicastRemoteObject;
7  import javax.rmi.ssl.SslRMIClientSocketFactory;
8  import javax.rmi.ssl.SslRMIServerSocketFactory;
```

```
9
10 public class Ejemplo implements Ejemplo_I {
11     private static final int RMI_PORT = 2099;
12
13     public Ejemplo() {
14         super();
15     }
16
17     public synchronized void escribir_mensaje(String mensaje) {
18         System.out.println("\nEntra Hebra " + mensaje);
19         //Buscamos los procesos 0, 10, 20,...
20         if (mensaje.endsWith("0")) {
21             try {
22                 System.out.println("Empezamos a dormir");
23                 Thread.sleep(5000);
24                 System.out.println("Terminamos de dormir");
25             } catch (Exception e) {
26                 System.err.println("Ejemplo exception:");
27                 e.printStackTrace();
28             }
29         }
30         System.out.println("Sale Hebra " + mensaje);
31     }
32
33     public static void main(String[] args) {
34         try {
35             // Definimos el nombre del objeto remoto que se registrará en el ligador
36             String nombre_objeto_remoto = "Ejemplo_I";
37
38             // Creamos una instancia del objeto remoto
39             Ejemplo_I prueba = new Ejemplo();
40
41             // Creamos los socket factories para SSL
42             SslRMIClientSocketFactory csf = new SslRMIClientSocketFactory();
43             SslRMIServerSocketFactory ssf = new SslRMIServerSocketFactory();
44
45             // Creamos el stub del objeto remoto para el cliente, exportandolo al
46             // puerto 0 (puerto anonimo) y usando los socket factories creados.
47             Ejemplo_I stub =
48                 (Ejemplo_I) UnicastRemoteObject.exportObject(prueba, 0, csf, ssf);
49
50             // El Registry vive como proceso externo (rmiregistry).
51             Registry registry = LocateRegistry.createRegistry(RMI_PORT);
52
53             // Registramos el objeto remoto en el ligador
54             registry.rebind(nombre_objeto_remoto, stub);
55
56             // A partir de aquí, el objeto remoto ya puede ser invocado por los clientes.
57
```

```
58              System.out.println("Ejemplo bound");
59          }
60          catch (Exception e) {
61              System.err.println("Ejemplo exception:");
62              e.printStackTrace();
63          }
64      }
65  }
```

2.2.3. `Cliente_Ejemplo_Multi_Threaded.java` (El Cliente)

Esta clase representa el cliente. Es distinta con respecto a la del ejercicio 1. Esta clase `Cliente_Ejemplo_Multi_Threaded` implementa la interfaz `Runnable` que nos permite crear *threads* o hebras en Java. Uno de los cambios es que ahora se le pasa el host del servidor como parámetro en el constructor de la clase, almacenándolo en un atributo. Como implementa esta interfaz la interfaz `Runnable`, tiene que darle una implementación al método `run()`, que será ejecutado por cada una de las hebras. En este método, cada hebra busca su propio stub y llama al servidor (al objeto remoto) de forma concurrente con el resto de hebras creadas.

En el `main`, se extrae el número de hebras, que se pasa como argumento al programa. Se crean tantas hebras como se hayan indicado, pasándoles el nombre del servidor al crearlas e indicándole su nombre, que será `Cliente + i`, donde i será un entero entre 0 y el número de hebras - 1. Las hebras se guardarán en un array.

Su código es:

```
1   // Cliente_Ejemplo_Multi_Threaded.java
2   // Código del cliente que invoca al objeto remoto implementado en Ejemplo.java
3
4   import java.rmi.registry.LocateRegistry;
5   import java.rmi.registry.Registry;
6
7   public class Cliente_Ejemplo_Multi_Threaded implements Runnable {
8       private static final int RMI_PORT = 2099;
9       public String nombre_objeto_remoto = "Ejemplo_I";
10      public String server;
11
12      public Cliente_Ejemplo_Multi_Threaded(String server) {
13          // Almacenamos el nombre del host del servidor
14          this.server = server;
15      }
16
17      public void run() {
18          System.out.println("Buscando el objeto remoto");
```

```
19          try {
20              Registry registry = LocateRegistry.getRegistry(server, RMI_PORT);
21              // Invocamos el objeto remoto obtenido del ligador
22              Ejemplo_I instancia_local = (Ejemplo_I) registry.lookup(nombre_objeto_remoto);
23              System.out.println("Invocando el objeto remoto");
24              // Llamamos al método remoto, pasándole
25              // el id del proceso (recibido como argumento)
26              instancia_local.escribir_mensaje(Thread.currentThread().getName());
27          } catch (Exception e) {
28              System.err.println("Ejemplo_I exception:");
29              e.printStackTrace();
30          }
31      }
32
33      public static void main(String args[]) {
34          int n_hebras = Integer.parseInt(args[1]);
35          Cliente_Ejemplo_Multi_Threaded[] v_clientes = new Cliente_Ejemplo_Multi_Threaded[n_hebr
36          Thread[] v_hebras = new Thread[n_hebras];
37          for (int i = 0; i < n_hebras; i++) {
38              //A cada hebra le pasamos el nombre el servidor
39              v_clientes[i] = new Cliente_Ejemplo_Multi_Threaded(args[0]);
40              v_hebras[i] = new Thread(v_clientes[i], "Cliente " + i);
41              v_hebras[i].start();
42          }
43      }
44  }
```

2.2.4. `launch.sh` (Script de Ejecución)

Simplemente se ha modificado para lanzar el servidor y el cliente correctamente. Al cliente se le indica que cree 12 hebras.

Las únicas líneas que han cambiado con respecto al ejemplo 1 son:

```
58
59  echo
60  echo "Lanzando el servidor Ejemplo 2"
61  java -cp . \
62          -Djava.rmi.server.hostname=localhost \
63          -Djavax.net.ssl.keyStore="$KEYSTORE" \
64          -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
65          -Djavax.net.ssl.keyStoreType=PKCS12 \
66          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
67          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
68          -Djavax.net.ssl.trustStoreType=PKCS12 \
69          Ejemplo &
70
71  sleep 2
```

```
72
73  echo
74  echo "Lanzando el cliente multi-hebra (12 hebras)"
75  java -cp . \
76          -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
77          -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
78          -Djavax.net.ssl.trustStoreType=PKCS12 \
79          Cliente_Ejemplo_Multi_Threaded localhost 12
```

2.2.5. Diagrama de clases del ejemplo 2.

El diagrama de clases en UML del ejemplo 2 es:

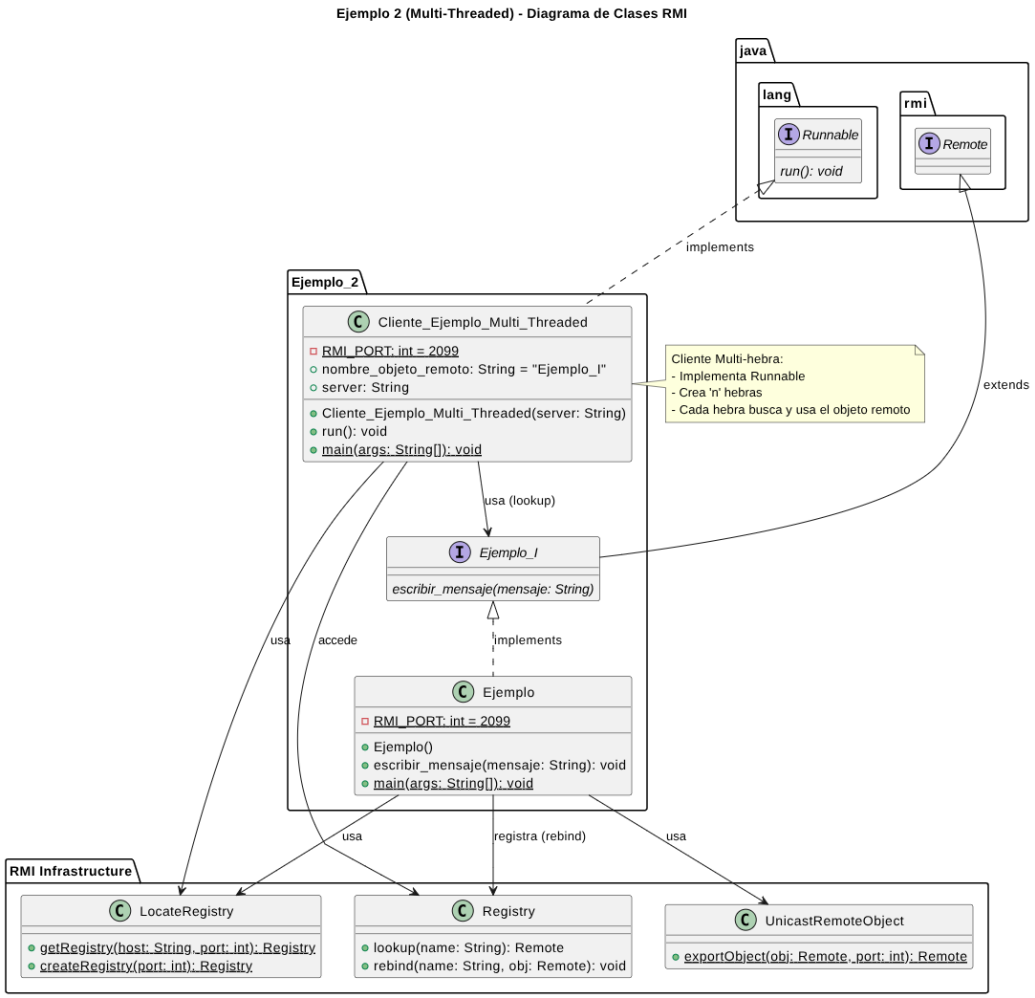


Figura 3: Diagrama de clases del ejemplo 2

2.2.6. Ejemplo de compilación y ejecución del ejemplo 2. Respondiendo a las preguntas

De nuevo, damos permiso de ejecución al script y lo ejecutamos. La salida sería:

```
angelrf@angel-ubuntu:~/Practica_3_RMI/codigo/Parte_1/Ejemplo_2$ chmod u+x launch.sh

angelrf@angel-ubuntu:~/Practica_3_RMI/codigo/Parte_1/Ejemplo_2$ ./launch.sh


Compilando con javac ...
Lanzando el servidor Ejemplo 2
Ejemplo bound
Lanzando el cliente multi-hebra (22 hebras)
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Buscando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto
Invocando el objeto remoto

Entra Hebra Cliente 2
Sale Hebra Cliente 2

Entra Hebra Cliente 8
Sale Hebra Cliente 8

Entra Hebra Cliente 9
Sale Hebra Cliente 9

Entra Hebra Cliente 7
Sale Hebra Cliente 7

Entra Hebra Cliente 5
Sale Hebra Cliente 5

Entra Hebra Cliente 4
Sale Hebra Cliente 4

Entra Hebra Cliente 10
Empezamos a dormir

Entra Hebra Cliente 0
Empezamos a dormir

Entra Hebra Cliente 11
Sale Hebra Cliente 11

Entra Hebra Cliente 6
Sale Hebra Cliente 6

Entra Hebra Cliente 3
Sale Hebra Cliente 3
Invocando el objeto remoto

Entra Hebra Cliente 1
Sale Hebra Cliente 1
Terminamos de dormir
Sale Hebra Cliente 0
Terminamos de dormir
Sale Hebra Cliente 10
```

Figura 4: 1ª ejecución del ejemplo 2

Nos piden que comprobemos que ocurre con las hebras cuyo nombre acaba en 0 y en que se diferencian del resto. Pues, como ya hemos comentado anteriormente, las hebras cuyo nombre es múltiplo de 10, el método remoto `escribir_mensaje` las duerme durante 5 segundos.

Luego nos indican que prestemos atención a la impresión de mensajes y comprobemos si estos mensajes se entrelazan. La respuesta es SÍ. Si nos fijamos en el ejemplo de ejecución que muestro arriba, podemos observar al final del todo que, las hebras 0 y 10 (que son las que tienen que dormir puesto que son las únicas que terminan en 0 de las 12 hebras que hay en total), entran al método pero se quedan dormidas, y el resto de hebras conforme entran al método, salen de forma instantánea. Mientras que, estas dos hebras, es al final, pasados esos 5 segundos, cuando despiertan y ya terminan. Con esto podemos ver como las hebras se ejecutan y llaman al método remoto (realizan peticiones/llamadas al servidor) de forma concurrente.

Vamos a hacer la **2ª modificación que nos pide el guion.** Voy a introducir el modificador `synchronized` en el método de la implementación remota y vamos a ver un ejemplo de ejecución y vamos a analizar las diferencias con respecto a la ejecución anterior.

¡Ojo! Esta modificación se realiza en la clase donde se implementa el método, NO en la interfaz donde se define. En nuestro caso, sería en `Ejemplo.java`. El método quedaría como:

```
public synchronized void escribir_mensaje (String mensaje)
throws RemoteException;
```

Volvemos a ejecutar el script con: `$ ./launch.sh` y vemos que su salida es:



Figura 5: 2ª ejecución del ejemplo 2

Vamos a explicar que esta sucediendo. Al añadir la palabra `synchronized` **estamos convertimos todo el método en una sección crítica. Java asocia a cada objeto un monitor o cerrojo (*lock*)** como los que estudiamos en la asignatura de *Sistemas Distribuidos y Concurrentes*. Entonces si hay varias clases que quieren acceder a una clase o método, sólo puede acceder una a la vez y modificarla, y así no pueden modificarlas todas de golpe. Es por eso que, cuando una hebra entra en el método remoto `escribir_mensaje` adquiere automáticamente el cerrojo de la instancia del objeto `Ejemplo`. Hasta que la hebra no abandona el método, no se libera el cerrojo. Es por eso que, cuando entra la hebra 0 o 10, y se ponen a dormir esos 5 segundos, las hebras que vienen detrás tendrán que esperar también esos 5 segundos (entrando en un estado bloqueo), hasta que la hebra múltiplo de 10 se despierte y abandone el método, liberando así el cerrojo.

2.3. Ejemplo 3. Contador.

En este ejemplo sobre un contador vamos a crear **por una parte el objeto remoto**, que está formado por varias funciones accesibles de forma remota; **y por otra parte el servidor**, implementado en el archivo `servidor.java`, que exporta los métodos contenidos en la interfaz `icontador.java` del objeto remoto instanciado como `micontador` de la clase definida en `contador.java`.

Por otro lado encontramos el **programa cliente**, que es una aplicación en Java y realiza las siguientes acciones: inicializar el contador del servidor, invocar el método `incrementar` del contador unas 1.000 veces, e imprimir el valor final del contador junto con el tiempo de respuesta medio que se ha calculado a partir de las invocaciones remotas del método incrementar.

De nuevo, comentar que los archivos han sido levemente modificados para evitar los problemas de componentes obsoletos. Vamos a verlos:

2.3.1. `icontador.java` (La Interfaz Remota)

Declara los métodos que tendrá el objeto remoto que la implemente.

```
1  // Fichero: icontador.java
2  // Define la Interfaz remota
3
4  import java.rmi.Remote;
5  import java.rmi.RemoteException;
6
```

```
7  public interface icontador extends Remote {
8
9      int sumar() throws RemoteException;
10
11     void sumar(int valor) throws RemoteException;
12
13     public int incrementar() throws RemoteException;
14 }
```

2.3.2. `contador.java` (El Objeto Remoto)

Es el objeto remoto que implementa la interfaz remota, y por lo tanto, le da una implementación a los métodos declarados en dicha interfaz. Con respecto al ejemplo que nos proporciona el guion, se ha quitado el `extends UnicastRemoteObject` para exportarlo manualmente en el servidor con SSL.

```
1  // Fichero: contador.java
2  // Es el objeto remoto que implementa la interfaz remota
3
4  import java.rmi.RemoteException;
5
6  public class contador implements icontador {
7      private int suma;
8
9      public contador() {
10         suma = 0;
11     }
12
13     public int sumar() throws RemoteException {
14         return suma;
15     }
16
17     public void sumar(int valor) throws RemoteException {
18         suma = valor;
19     }
20
21     public int incrementar() throws RemoteException {
22         suma++;
23         return suma;
24     }
25 }
```

2.3.3. `servidor.java` (El Servidor)

Es el servidor encargado de publicar el objeto remoto anterior.

```java
// Fichero: servidor.java
// Es el servidor que publica el objeto remoto contador

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.rmi.server.UnicastRemoteObject;
import javax.rmi.ssl.SslRMIClientSocketFactory;
import javax.rmi.ssl.SslRMIServerSocketFactory;

public class servidor {
    public static void main(String[] args) {
        try {
            int puerto = 1099;
            contador micontador = new contador();

            // Configuramos las factorías SSL
            SslRMIClientSocketFactory csf = new SslRMIClientSocketFactory();
            SslRMIServerSocketFactory ssf = new SslRMIServerSocketFactory();

            // Creamos el stub del objeto remoto para el cliente, exportándolo
            // al puerto 0 (puerto anónimo) y usando las factorías SSL
            icontador stub = (icontador) UnicastRemoteObject.exportObject(micontador, 0, csf,

            // Creamos el registro en el puerto 1099
            Registry registry = LocateRegistry.createRegistry(puerto);
            registry.rebind("mmicontador", stub);

            System.out.println("Servidor contador preparado con SSL en puerto " + puerto);
        } catch (Exception e) {
            System.err.println("Excepción en el servidor: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

2.3.4. `cliente.java` (El Cliente)

Implementa el cliente. Sus funciones ya se comentaron anteriormente.

```java
// Fichero: cliente.java
// Es el cliente que invoca al objeto remoto contador

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class cliente {
    public static void main(String[] args) {
```

```
9           if (args.length < 1) {
10              System.out.println("Uso: java cliente <host>");
11              System.exit(1);
12          }
13
14          try {
15              // Crea el stub para el cliente especificando el nombre del servidor
16              // y el puerto del registro RMI
17              Registry mireg = LocateRegistry.getRegistry(args[0], 1099);
18
19              icontador micontador = (icontador) mireg.lookup("mmicontador");
20
21              // Pone el contador al valor inicial 0
22              System.out.println("Poniendo contador a 0...");
23              micontador.sumar(0);
24
25              // Obtiene hora de comienzo
26              long horacomienzo = System.currentTimeMillis();
27
28              // Incrementa 1000 veces
29              System.out.println("Incrementando 1000 veces...");
30              for (int i = 0; i < 1000; i++) {
31                  micontador.incrementar();
32              }
33              // Obtiene hora final, realiza e imprime calculos
34              long horafin = System.currentTimeMillis();
35              System.out.println("Media de las RMI realizadas = "
36                              + ((horafin - horacomienzo) / 1000f)
37                              + " msegs");
38              System.out.println("Valor final del contador = " + micontador.sumar());
39
40          } catch (Exception e) {
41              System.err.println("Excepción del sistema: " + e);
42          }
43          System.exit(0);
44      }
45  }
```

2.3.5. `launch.sh` (Script para Ejecución)

Se ha modificado la forma en la que se lanza el cliente y el servidor. Las únicas líneas que han cambiado con respecto a los ejemplos anteriores son:

```
58
59  echo "Lanzando Servidor Contador (Ejemplo 3)"
60  java -cp . \
```

```
61      -Djava.rmi.server.hostname=localhost \
62      -Djavax.net.ssl.keyStore="$KEYSTORE" \
63      -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
64      -Djavax.net.ssl.keyStoreType=PKCS12 \
65      -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
66      -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
67      -Djavax.net.ssl.trustStoreType=PKCS12 \
68      servidor &
69
70  sleep 2
71
72  echo "Lanzando Cliente Contador"
73  java -cp . \
74      -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
75      -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
76      -Djavax.net.ssl.trustStoreType=PKCS12 \
77      cliente localhost
```

2.3.6. Diagramas en UML del ejemplo 3.

El **diagrama de clases** en UML de este ejemplo es:

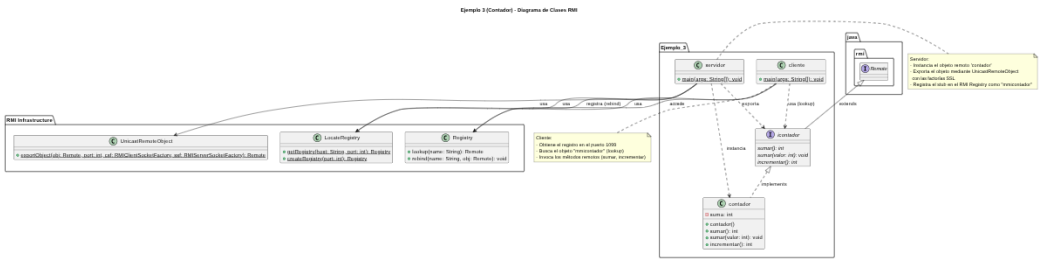


Figura 6: Diagrama de clases en UML del ejemplo 3

Y también muestro a continuación el **diagrama de secuencia** en UML de este ejemplo, ya que considero qué es interesante ya que hemos separado el objeto remoto del servidor.

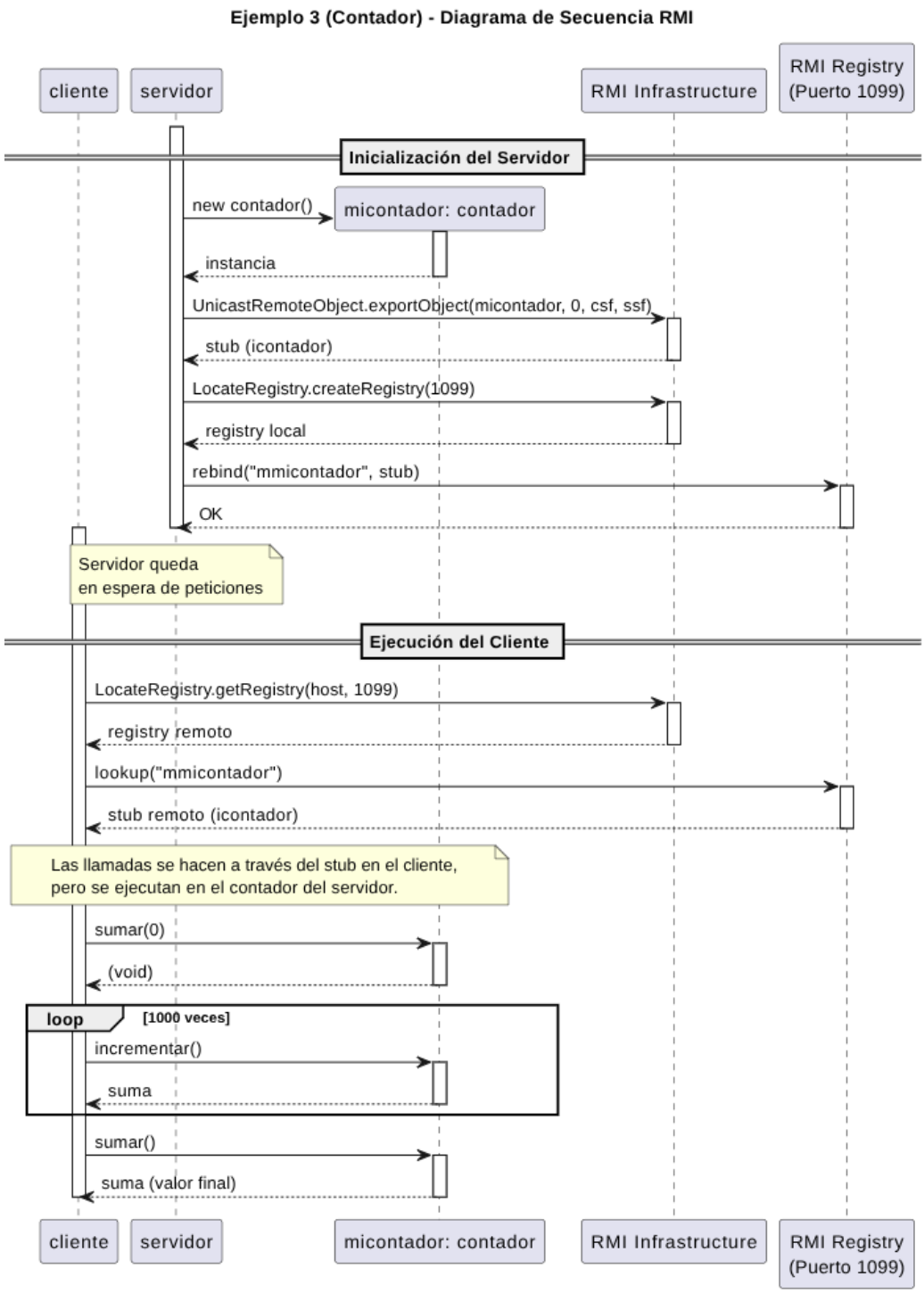


Figura 7: Diagrama de secuencia en UML del ejemplo 3

2.3.7. Ejemplo de compilación y ejecución del ejemplo 3.

Una vez más, al igual que hicimos en los ejemplos anteriores, damos permiso de ejecución al script y lo ejecutamos.

La salida sería:

```
angelrf@angel-ubuntu:~/Practica_3_RMI/codigo/Parte_1/Ejemplo_3$ chmod u+x launch.sh
angelrf@angel-ubuntu:~/Practica_3_RMI/codigo/Parte_1/Ejemplo_3$ ./launch.sh

Generando keystore TLS de ejemplo ...
Generando par de claves RSA de 2.048 bits para certificado autofirmado (SHA384withRSA) con una
 validez de 3.650 días
        para: CN=localhost, OU=RMI, O=DSD, L=Local, ST=Local, C=ES

Generando truststore TLS de ejemplo ...
Certificado almacenado en el archivo <rmi-server.crt>
Se ha agregado el certificado al almacén de claves

Compilando con javac ...
Lanzando Servidor Contador (Ejemplo 3)
Servidor contador preparado con SSL en puerto 1099
Lanzando Cliente Contador
Poniendo contador a 0...
Incrementando 1000 veces...
Media de las RMI realizadas = 0.233 msegs
Valor final del contador = 1000
```

Figura 8: Ejecución del ejemplo 3

Vamos a comentar varias cosas que podemos deducir tras ver el código del ejemplo y la imagen de la salida (Figura 8):

- **El servidor recuerda el valor de suma.** Si lanzamos el cliente varias veces sin reiniciar el servidor (y quitamos la línea de `micontador.sumar(0)`), podremos ver que el contador sigue subiendo. El objeto remoto vive en el servidor mientras este esté en ejecución.
- Al realizar 1000 llamadas seguidas, **el cliente calcula el tiempo medio de una invocación RMI**, que lo podemos ver en la figura 8. **Llamar al método remoto es mucho más lento que una llamada a un método local**, porque cada incremento implica:
  - Serializar el nombre del método.
  - Enviar el paquete por la red (TCP/SSL).
  - Procesarlo en el servidor.
  - Recibir la confirmación o respuesta.
- Vemos que hay una **gran transparencia**, ya que el cliente usa el método `incrementar()` como si fuera un objeto local, pero el cálculo real ocurre en la JVM del servidor.

3. Parte 2. Servidor Replicado.

3.1. Introducción y Objetivos.

En esta segunda parte de la práctica, teníamos el objetivo de diseñar y programar aplicaciones distribuidas siguiendo el Cliente-Servidor y, aunque estas aplicaciones son sencillas, nos permiten abordar los aspectos más importates a tener en cuenta en en el desarrollo de este tipo de aplicaciones. En concreto, el objetivo era que **desarrollar con Java RMI un sistema cliente-servidor para realizar donaciones monetarias para una causa humanitaria, con las siguientes características**:

- El servidor sería un servidor replicado y sería el encargado de de recibir donaciones de entidades (clientes) para la causa.
- El servidor proporcionaría en principio dos operaciones, registro de una entidad interesada (cliente) en la causa, y depósito de una donación a la causa, siendo imposible realizar un depósito (o más) sin haberse registrado como cliente (o donante) previamente.
- Cuando un donante deseaba registrarse y contactaba con cualquiera de las dos réplicas del servidor para hacerlo, entonces el registro del donante debía ocurrir de forma real y transparente en la réplica con menos entidades registradas en ese momento. Es decir, el donante se dirigió a una réplica aunque esta no sea donde ha quedado registrado finalmente, pero a partir de ese momento, el donante realizará los depósitos en la réplica del servidor donde fue registrado.
- Cada réplica del servidor almacena y actualiza el subtotal de las donaciones realizadas en dicha replica.
- Un cliente no puede registrarse más de una vez, ni siquiera en replicas distintas.
- Los servidores también ofrecen, en principio, dos operaciones de consulta: total donado y listado de donantes. Dichas operaciones sólo se llevarán a cabo si el cliente/donante previamente se ha registrado y ha realizado al menos un depósito. Cuando un cliente realiza alguna de estas consultas, solamente podrá hacer la petición a la réplica donde se encuentra registrado, y esta réplica es la encargada de devolver el resultado después de solicitar la información oportuna a la otra réplica.

Estas son las **características básicas que nos pedía el guion, y que el sistema tiene en cuenta, implementándolas correctamente**. Además, **se han añadido más mejoras que se nos indicaban en el archivo** `normas.pdf`, las cuáles son:

- **Se han añadido más operaciones al servidor.**
- Se pueden utilizar más de 2 réplicas. En concreto, **podemos utilizar el número de réplicas que queramos**, ya que se crean y se comunican en tiempo de ejecución perfectamente (lo veremos más en detalle cuando veamos el servidor, más adelante).

3.2. Arquitectura del Sistema.

A continuación vamos a ver los componentes del sistema, además del diagrama de clases en UML y el diagrama de secuencia con un par de operaciones de ejemplo.

3.2.1. `iDonacion.java` (La Interfaz Remota)

Es la interfaz remota, que sería el çontrato.[en]tre el cliente y el servidor, ya que hereda de `Remote` y es conocida por estos dos componentes para que el *stub* sepa los métodos que pueden ser invocados a través de la red. Estos métodos devuelven tipos básicos (`int`, `String`) o incluso referencias a otros objetos remotos (`iDonacion`), lo que permite al cliente saltar de un servidor a otro de forma transparente.

El código de este componente es:

```java
/**
 * @file iDonacion.java
 * @brief Define la interfaz remota iDonacion.
 * Define que pueden hacer los clientes y que pueden pedirse los servidores entre ellos.
 */

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface iDonacion extends Remote {

    /**
     * @brief Devuelve el nombre del nodo, que es su identificador único.
     * Esto es útil para que los servidores puedan conocerse entre ellos y replicarse.
     * @return String. El nombre del nodo del servidor.
     * @throws RemoteException Si ocurre un error de comunicación remota.
     */
    String getNombre() throws RemoteException;

    /**
     * @brief Permite que un servidor conozca a otro servidor (peer) para que
```

```
22          * puedan replicarse entre ellos.
23          * @param peer El servidor peer que se va a conocer.
24          * @throws RemoteException Si ocurre un error de comunicación remota.
25          */
26             void setPeer(iDonacion peer) throws RemoteException;
27
28         /**
29          * @brief Permite al cliente consultar el total de donantes registrados.
30          * @return int. El total de donantes registrados.
31          * @throws RemoteException Si ocurre un error de comunicación remota.
32          */
33         int totalDonantes() throws RemoteException;
34
35         /**
36          * @brief Permite al cliente consultar si un donante está registrado.
37          * @param donante Nombre del donante a consultar.
38          * @return boolean. true si el donante está registrado, false en caso contrario.
39          * @throws RemoteException Si ocurre un error de comunicación remota.
40          */
41         boolean estaRegistrado(String donante) throws RemoteException;
42
43         /**
44          * @brief Devuelve el número total de donaciones realizadas en esta réplica.
45          * @return int. El número total de donaciones realizadas.
46          * @throws RemoteException Si ocurre un error de comunicación remota.
47          */
48         int getTotalDonacionesLocal() throws RemoteException;
49
50         /**
51          * @brief Devuelve una lista con los nombres de todos los donantes registrados
52          * en esta réplica.
53          * @return List<String>. Una lista con los nombres de todos los donantes registrados.
54          * @throws RemoteException Si ocurre un error de comunicación remota.
55          */
56         java.util.List<String> listarDonantesLocal() throws RemoteException;
57
58
59         //============================================================================
60         //                          ACCIONES QUE PUEDEN INVOCAR LOS CLIENTES
61         //============================================================================
62
63         /**
64          * @brief Permite a un cliente registrar un donante. Si el donante ya estaba
65          * registrado, no hace nada.
66          * @param donante Nombre del donante a registrar.
67          * @return iDonacion. Devuelve una referencia al servidor donde se ha registrado
68          *  el donante, para que el cliente pueda invocar a sus métodos.
69          * @throws RemoteException Si ocurre un error de comunicación remota.
70          */
71         iDonacion registrar(String donante) throws RemoteException;
72
73         /**
74          * @brief Permite dar de baja a un donante.
75          * @pre El donante debe estar registrado previamente.
76          * @param donante El nombre del donante a dar de baja.
77          * @throws RemoteException Si ocurre un error de comunicación remota.
78          */
79         void darDeBaja(String donante) throws RemoteException;
80
81         /**
82          * @brief Permite a un cliente registrar un donante y realizar una donación.
83          * @param donante El nombre del donante (debe estar registrado previamente).
```

```
84          * @param cantidad La cantidad a donar.
85          * @throws RemoteException Si ocurre un error de comunicación remota.
86          */
87         void donar(String donante,int cantidad) throws RemoteException;
88
89
90         //============================================================================
91         //                                      OPERACIONES DE CONSULTA
92         //============================================================================
93
94         /**
95              * @brief Permite al cliente consultar el total de donaciones realizadas por
96          * todos los donantes en todo el sistema.
97              * @pre El donante debe estar registrado previamente en ESTA réplica y haber
98          * realizado al menos una donación.
99              * @param donante El nombre del donante a consultar.
100             * @return int. El total de donaciones realizadas todo el sistema (en todas las réplicas).
101             * @throws RemoteException Si ocurre un error de comunicación remota.
102             */
103        int totalDonaciones(String donante) throws RemoteException;
104
105        /**
106         * @brief Permite al cliente listar todos los donantes registrados.
107         * @param donante Nombre del donante (debe estar registrado previamente).
108         * @return List<String>. Una lista con los nombres de todos los donantes registrados.
109         * @throws RemoteException Si ocurre un error de comunicación remota.
110         */
111        java.util.List<String> listarDonantes(String donante) throws RemoteException;
112
113        /**
114         * @brief Devuelve el total de donaciones realizadas por un donante específico
115         * en todo el sistema, sumando las donaciones realizadas.
116         * @pre El donante debe estar registrado previamente.
117         * @param donante El nombre del donante
118         * @return int. El total de donaciones realizadas por el donante en todo el sistema.
119         * @throws RemoteException
120         */
121        int getTotalDonacionesDonante(String donante) throws RemoteException;
122
123        /**
124         * @brief Devuelve el nombre del donante que ha donado la mayor cantidad de dinero
125         * desde que se registró en el sistema. Si hay varios donantes con la misma cantidad,
126         * se devuelve el que primero se encuentre en la lista de donantes registrados.
127         * @return String. El nombre del donante.
128         * @throws RemoteException Si ocurre un error de comunicación remota.
129         */
130        String mayorDonanteGlobal() throws RemoteException;
131
132        /**
133         * @brief Permite a un donante consultar su historial de donaciones.
134         * @pre El donante debe estar registrado previamente y haber realizado al menos
135         * una donación.
136         * @param donante El nombre del donante que quiere consultar su historial.
137         * @return List<Integer>. Lista con las cantidades donadas.
138         * @throws RemoteException Si ocurre un error de comunicación remota.
139         */
140        java.util.List<Integer> obtenerHistorialDonaciones(String donante) throws RemoteException;
141    }
```

3.2.2. `Donacion.java` (La Lógica de Negocio)

Como indica su título, se trata de la clase que implementa la lógica de negocio, tratándose posiblemente del archivo más importante de toda la práctica. La clase `Donacion` implementa la interfaz `iDonacion`. Vamos a comentar los aspectos más importantes de este componente:

- Tiene como atributos
  - `nombreNodo` que es el nombre o identificador de este nodo/réplica del servidor.
  - `totalDonacionesLocal` el total de donaciones realizadas de forma local en esta réplica.
  - `donantes` que es un `HashMap` que almacena los donantes registrados en esta réplica, junto con una lista de las cantidades donadas.
  - `peers` que es un `HashMap` que almacena las referencias a las otras réplicas del servidor. Los pares son: `<nombreNodo, iDonacion>`, es decir, el nombre del nodo y la referencia al objeto remoto de ese nodo.
- En el constructor de la clase, se pasa como parámetro el nombre del nodo.
- En el resto de métodos, que están definidos en la interfaz remota, se implementa la lógica de negocio y **se tiene en cuenta la gestión de la concurrencia, ya que se han añadido los modificadores** `synchronized` a los métodos que modifican el estado del objeto, para evitar así problemas de concurrencia; esto afecta al método entero o a una parte de este, ya que la he considerado como una sección crítica. Además, en algunos métodos, se realizan llamadas a otros objetos remotos (a otras réplicas del servidor) para solicitar información o realizar alguna acción, lo que hace que el sistema sea completamente funcional y transparente, haciendo una buena **gestión de la interoperabilidad entre réplicas**.
- Veámos el resto de métodos, empezando por las operaciones básicas que puede realizar el cliente, que son las siguientes:
  - `registrar` es el método que se encarga de registrar a un nuevo donante. Para ello, primero comprueba si el donante ya está registrado en esta réplica o en alguna de las otras réplicas, y si no

lo está, **lo registra en la réplica con menor número de donantes, haciendo así un Balanceo de Carga**. Esto permite que un servidor no solo registre, sino que decida la réplica más adecuada para registrar al cliente. Además, la parte de código que se encarga del registro se implementa como una sección crítica, añadiendo el modificador `synchronized` para evitar problemas de concurrencia.

A la hora de implementar este método me surgió un error, y es que, tras ejecutar todo el proyecto, registré un donante en una réplica, y al intentar realizar otro registro desde el mismo cliente me ocurrió un **intebloqueo (o *dearlock*)** debido a que, cuando entraba en el método `registrar` para comprobar si el donante ya estaba registrado, se llamaba a la otra réplica para realizar la misma comprobación, y esta réplica a su vez llamaba a la primera réplica para realizar la misma comprobación, y así sucesivamente, quedando ambas réplicas bloqueadas esperando una respuesta de la otra. Para solucionar este problema, eliminé el modificador `synchronized` de la cabecera del método `registrar`, y solamente lo añadí a la parte del código que debían ejecutarse en exclusión mutua. **Esto me sirvió para tenerlo en cuenta en otros métodos, como `totalDonaciones`, que también tenía el mismo problema.**

- `darDeBaja` es el método que se encarga de dar de baja a un donante registrado. Para realizar esta acción, primero **comprueba si el donante está registrado en esta réplica, y si lo está, elimina su registro y actualiza el total de donaciones local**.
- `donar` es el método que se encarga de realizar una donación. Para ello, primero comprueba si el donante está registrado en esta réplica, y si lo está, añade la cantidad donada a su lista de donaciones y actualiza el total de donaciones local.

- En cuanto a las operaciones de consulta, tenemos las siguientes:
  - `totalDonaciones` es el método que se encarga de calcular el total donado por un donante. Para ello, primero **comprueba si el donante está registrado en el sistema**, no sólo en la réplica, ya que el objetivo de este método es consultar cualquier donante independientemente de la réplica en la que esté registrado. Si está registrado, suma todas las cantidades donadas por ese donante y devuelve el resultado. Cuando se accede a comprobar si el donante está registrado en el sistema, esa porción de código se implementa

como una sección crítica, añadiendo el modificador `synchronized` para evitar que otro cliente pueda modificar el estado del objeto mientras se está realizando esta comprobación.

- `listadoDonantes` es el método que se encarga de devolver un listado con los nombres de los donantes registrados en todo el sistema.
- `getTotalDonacionesDonante` es el método que se encarga de devolver el total donado por un donante concreto. Para ello, primero comprueba si el donante está registrado en el sistema, y si lo está, devuelve el total de donaciones local de ese donante.
- `mayorDonanteGlobal` es el método que se encarga de devolver el nombre del donante que más ha donado en todo el sistema. Para ello, recorre la lista de donantes de esta réplica y de las otras réplicas para encontrar el donante con mayor total de donaciones.
- `obtenerHistorialDonaciones` es el método que se encarga de devolver el historial de donaciones de un donante concreto. Para ello, primero comprueba si el donante está registrado en el sistema, y si lo está, devuelve la lista de todas las cantidades que ha ido donando.

- El resto de métodos serían:

  - `getNombre` es el método que se encarga de devolver el nombre del nodo.
  - `setPeer` es el método que se encarga de añadir una referencia a otra réplica del servidor. Para ello, se le pasa como parámetro la referencia al objeto remoto de la otra réplica, y se añade a la lista de pares de esta réplica.
  - `totalDonantes` es el método que se encarga de devolver el número de donantes registrados en esta réplica.
  - `estaRegistrado` es el método que se encarga de comprobar si un donante está registrado en esta réplica. Devuelve un valor booleano indicando si el donante está registrado o no.
  - `getTotalDonacionesLocal` es el método que se encarga de devolver el total de donaciones (en €)realizadas en esta réplica.
  - `listarDonantesLocal` es el método que se encarga de devolver un listado con los nombres de los donantes registrados en esta réplica.

  El código de este componente es:

```java
/**
 * @file Donacion.java
 * @brief Implementación de la interfaz remota iDonacion. Este es el objeto remoto que
 * se va a publicar en el RMI Registry y que los clientes van a invocar.
 */

import java.rmi.*;
import java.util.ArrayList;
import java.util.List;


public class Donacion implements iDonacion {

        // Identificador único de la réplica del servidor
    private final String nombreNodo;

        // Total de donaciones realizadas de forma local (en esta réplica).
    private int totalDonacionesLocal = 0;

        // Map con los donantes registrados EN ESTA RÉPLICA y la cantidad total que han donado.
        // los pares son: <nombreDonante, cantidadDonacionesRealizadas>
    private final java.util.Map<String, List<Integer>> donantes = new java.util.HashMap<>();

        // Map con los peers (otras réplicas del servidor) que esta réplica conoce.
        private final java.util.Map<String, iDonacion> peers = new java.util.HashMap<>();

        /**
         * @brief Constructor de la clase Donacion.
         * @param nombreNodo El nombre del nodo, que es su identificador único.
         * Este nombre se genera dinámicamente en el servidor para asegurar que cada
         * instancia tenga un identificador único.
         */
    public Donacion(String nombreNodo) {
        this.nombreNodo = nombreNodo;
    }

        /**
         * @brief Devuelve el nombre del nodo, que es su identificador único.
         * Esto es útil para que los servidores puedan conocerse entre ellos y replicarse.
         *
         * @return El nombre del nodo del servidor.
         * @throws RemoteException Si ocurre un error de comunicación remota.
         */
        @Override
    public String getNombre() throws RemoteException {
        return nombreNodo;
    }

        /**
         * @brief Permite que un servidor conozca a otro servidor (peer) para que
         * puedan replicarse entre ellos. Cada servidor mantiene una lista de sus peers
         * y se actualizan mutuamente cuando se registran nuevos donantes o se realizan
         * nuevas donaciones.
         *
         * @param peer El servidor peer que se va a conocer.
         * @throws RemoteException Si ocurre un error de comunicación remota.
         */
        @Override
        public synchronized void setPeer(iDonacion peer) throws RemoteException {
                peers.put(peer.getNombre(), peer);
        }
```

```
62
63          /**
64           * @brief Devuelve el número total de donantes registrados en esta réplica.
65           * @return int. El número total de donantes registrados.
66           * @throws RemoteException Si ocurre un error de comunicación remota.
67           */
68          @Override
69          public synchronized int totalDonantes() throws RemoteException {
70                  return donantes.size();
71      }
72
73          /**
74           * @brief Permite consultar si un donante está registrado en esta réplica.
75           * @param donante Nombre del donante a consultar.
76           * @return boolean. true si el donante está registrado, false en caso contrario.
77           * @throws RemoteException Si ocurre un error de comunicación remota.
78           */
79          @Override
80          public synchronized boolean estaRegistrado(String donante) throws RemoteException {
81                  return donantes.containsKey(donante);
82          }
83
84          /**
85       * @brief Devuelve el número total de donaciones realizadas en esta réplica.
86       * @return int. El número total de donaciones realizadas.
87       * @throws RemoteException Si ocurre un error de comunicación remota.
88       */
89          @Override
90          public synchronized int getTotalDonacionesLocal() throws RemoteException {
91                  return totalDonacionesLocal;
92          }
93
94          /**
95           * @brief Devuelve una lista con los nombres de todos los donantes registrados en esta réplica.
96           * @return List<String>. Una lista con los nombres de todos los donantes registrados.
97           * @throws RemoteException Si ocurre un error de comunicación remota.
98           */
99          @Override
100         public synchronized List<String> listarDonantesLocal() throws RemoteException {
101                 return new ArrayList<>(donantes.keySet());
102         }
103
104
105
106
107         //============================================================================
108         //                          ACCIONES QUE PUEDEN INVOCAR LOS CLIENTES
109         //============================================================================
110
111         /**
112          * @brief Permite registrar un donante.
113          *
114          * @param donante El nombre del donante a registrar.
115          * @return iDonacion. El servidor (réplica) donde se ha registrado el donante.
116          * @throws RemoteException Si ocurre un error de comunicación remota.
117          */
118         @Override
119         public iDonacion registrar(String donante) throws RemoteException {
120                 iDonacion peerRegistrado = null; // Réplica donde se ha registrado el donante
121
122                 // 1. Comprobar si el donante ya está registrado en esta réplica.
123                 if(this.estaRegistrado(donante)) {
```

```
124                         peerRegistrado = this; // El donante ya está registrado en esta réplica
125                 } else {
126                         // 2. Comprobamos si no esta registrado en alguna de las otras réplicas
127                         for(iDonacion peer : peers.values()) {
128                                 if(peer.estaRegistrado(donante)) {
129                                         peerRegistrado = peer;
130                                         break;
131                                 }
132                         }
133                 }
134 
135                 // Si el donante no está registrado en ninguna réplica, lo registramos en
136                 // la réplica con menos donantes registrados para balancear la carga entre las réplicas.
137                 if(peerRegistrado == null) {
138 
139                         // 3. Encontramos la réplica con menos donantes registrados
140                         iDonacion mejorPeer = this;         // Mejor réplica encontrada para registrar el nuevo donant
141                         int minDonantes = this.totalDonantes(); // Minimo nro de donantes registrado en la mejorPeer
142 
143                         for(iDonacion peer : peers.values()) {
144                                 int nroDonantesPeer = peer.totalDonantes();
145                                 if(nroDonantesPeer < minDonantes) {
146                                         mejorPeer = peer;
147                                         minDonantes = nroDonantesPeer;
148                                 }
149                         }
150 
151                         // 4. Registramos el nuevo donante en la mejor réplica encontrada
152                         if(mejorPeer.getNombre().equals(this.getNombre())) {
153                                 synchronized(this) { // Lo hacemos de exclusión mutua para evitar condiciones de carr
154                                         donantes.putIfAbsent(donante, new ArrayList<>()); // Registramos de forma seg
155                                 }
156                                 peerRegistrado = this;
157                         } else {
158                                 // Registramos el nuevo donante en la mejor réplica encontrada
159                                 // Como este método ya no bloquea globalmente, evitamos el deadlock
160                                 peerRegistrado = mejorPeer.registrar(donante);
161                         }
162                 }
163 
164                 // El cliente obtiene una referencia al stub del servidor donde se acaba
165                 // de registrar el donante, y así puede invocar a sus métodos.
166                 return peerRegistrado;
167         }
168 
169             /**
170          * @brief Permite dar de baja a un donante.
171          * @pre El donante debe estar registrado previamente.
172          * @param donante El nombre del donante a dar de baja.
173          * @throws RemoteException Si ocurre un error de comunicación remota.
174          */
175             @Override
176         public synchronized void darDeBaja(String donante) throws RemoteException{
177                 // Comprobamos que el donante esté registrado en esta réplica
178                 if(this.estaRegistrado(donante)){
179                         donantes.remove(donante);
180                         System.out.println("[INFO.] El donante " + donante
181                                 + " ha sido dado de baja en " + this.getNombre() + ".");
182                 } else {
183                         System.out.println("[ERROR.] El donante " + donante
184                                 + " ha intentado darse de baja en " + this.getNombre()
185                                 + " pero no está registrado en esta réplica.");
```

```
186                 }
187         }
188
189         /**
190          * @brief Permite a un donante realizar una donación.
191          * @pre El donante debe estar registrado previamente en ESTA réplica.
192          * @param donante El nombre del donante que realiza la donación.
193          * @param cantidad La cantidad a donar.
194          * @throws RemoteException Si ocurre un error de comunicación remota.
195          */
196         @Override
197     public synchronized void donar(String donante, int cantidad) throws RemoteException {
198                 // Comprobamos que el donante esté registrado en esta réplica
199                 if(this.estaRegistrado(donante)){
200                         totalDonacionesLocal += cantidad;
201                         donantes.get(donante).add(cantidad); // Añadimos la cantidad donada al historial del donante
202                         System.out.println("[INFO.] El donante " + donante
203                                 + " ha realizado una donación de " + cantidad
204                                 + "€");
205                 } else {
206                         System.out.println("[ERROR.] El donante " + donante
207                                 + " ha intentado realizar una donación"
208                                  + " en " + this.getNombre()
209                                 + " pero no está registrado en esta réplica.");
210                 }
211     }
212
213
214         //=============================================================================
215         //                                      OPERACIONES DE CONSULTA GLOBAL
216         //=============================================================================
217
218         /**
219          * @brief Permite al cliente consultar el total de donaciones realizadas por todos los donantes en todo el sis
220          * @pre El donante debe estar registrado previamente en ESTA réplica y haber realizado al menos una donación.
221          * @param donante El nombre del donante a consultar.
222          * @return int. El total de donaciones realizadas todo el sistema (en todas las réplicas).
223          * @throws RemoteException Si ocurre un error de comunicación remota.
224          */
225         @Override
226     public int totalDonaciones(String donante) throws RemoteException {
227                 int totalDonacionesGlobal = 0;
228                 boolean tienePermiso = false;
229
230                 //1. Hacemos de forma sincronizada sólamente la comprobación local
231                 synchronized(this) {
232                         if(this.estaRegistrado(donante) && !donantes.get(donante).isEmpty()) {
233                                 tienePermiso = true;
234                         }
235                 }
236
237                 // 2. Si el donante cumple las precondiciones para consultar...
238                 if(tienePermiso) {
239                         // 3. Sumamos las donaciones realizadas en esta réplica.
240                         totalDonacionesGlobal += this.getTotalDonacionesLocal();
241
242                         // 4. Sumamos las donaciones realizadas en las otras réplicas.
243                         for(iDonacion peer : peers.values()) {
244                                 totalDonacionesGlobal += peer.getTotalDonacionesLocal();
245                         }
246                 } else {
247                         // Si el donante no cumple las precondiciones, mostramos un mensaje de
```

```
248                         // error indicando el motivo y de forma síncrona.
249                         synchronized(this) {
250                                 if(!this.estaRegistrado(donante)) {
251                                         System.out.println("[ERROR.] El donante " + donante
252                                                 + " no está registrado en la réplica:" + this.getNombre());
253                                 } else {
254                                         System.out.println("[ERROR.] El donante " + donante
255                                                 + " no ha realizado ninguna donación.");
256                                 }
257                         }
258                         totalDonacionesGlobal = -1;
259                 }
260
261                 return totalDonacionesGlobal;
262         }
263
264             /**
265              * @brief Permite al cliente listar todos los donantes registrados en el sistema.
266              * @pre El donante debe estar registrado previamente en ESTA réplica y haber realizado al menos una donación.
267              * @param donante El nombre del donante que realiza la consulta.
268              * @return List<String>. Una lista con los nombres de todos los donantes registrados.
269              * @throws RemoteException Si ocurre un error de comunicación remota.
270              */
271             @Override
272         public List<String> listarDonantes(String donante) throws RemoteException {
273
274                 List<String> donantesRegistradosGlobal = null;
275                 boolean tienePermiso = false;
276
277                 // 1. Hacemos de forma sincronizada sólamente la comprobación local
278                 synchronized(this) {
279                         if(this.estaRegistrado(donante) && !donantes.get(donante).isEmpty()) {
280                                 tienePermiso = true;
281                         }
282                 }
283
284                 // 2. Si el donante cumple las precondiciones para consultar...
285                 if(tienePermiso) {
286                         // 3. Obtenemos los donantes registrados en esta réplica.
287                         donantesRegistradosGlobal = this.listarDonantesLocal();
288
289                         // 4. Obtenemos los donantes registrados en las otras réplicas.
290                         for(iDonacion peer : peers.values()) {
291                                 donantesRegistradosGlobal.addAll(peer.listarDonantesLocal());
292                         }
293                 } else {
294                         // Si el donante no cumple las precondiciones, mostramos un mensaje de
295                         // error indicando el motivo y de forma síncrona.
296                         synchronized(this) {
297                                 if(!this.estaRegistrado(donante)) {
298                                         System.out.println("[ERROR.] El donante " + donante
299                                                 + " no está registrado en la réplica:" + this.getNombre());
300                                 } else {
301                                         System.out.println("[ERROR.] El donante " + donante
302                                                 + " no ha realizado ninguna donación.");
303                                 }
304                         }
305                 }
306
307                 return donantesRegistradosGlobal;
308         }
309
```

```
310         /**
311          * @brief Devuelve el total de donaciones realizadas por un donante específico en todo el sistema,
312          * sumando las donaciones realizadas.
313          * @pre El donante debe estar registrado previamente.
314          * @param donante El nombre del donante
315          * @return int. El total de donaciones realizadas por el donante en todo el sistema.
316          * @throws RemoteException
317          */
318         @Override
319        public int getTotalDonacionesDonante(String donante) throws RemoteException {
320                 int total = 0;
321                 boolean encontrado = false;
322
323                 // 1. Comprobamos de forma segura si el donante está en esta réplica
324                 synchronized(this) {
325                         encontrado = this.estaRegistrado(donante);
326                         if (encontrado) { // Está registrado en esta réplica.
327                                 for (Integer cantidad : donantes.get(donante)) {
328                                         total += cantidad;
329                                 }
330                         }
331                 }
332
333                 if(!encontrado) {
334                         // 2. Si no está en esta réplica local, buscamos en qué réplica (peer) está registrado
335                         for (iDonacion peer : peers.values()) {
336                                 if (peer.estaRegistrado(donante)) {
337                                         // Si el donante está registrado en esta réplica peer,
338                                         // consultamos su total de donaciones en esa réplica peer
339                                         total = peer.getTotalDonacionesDonante(donante);
340                                         encontrado = true;
341                                         break;
342                                 }
343                         }
344                 }
345
346                 // Si llegados este punto no lo ha encontrado, es que no está registrado
347                 if(!encontrado)
348                         System.out.println("[ERROR.] El donante " + donante + " no está registrado en el sistema.");
349
350                 return total;
351         }
352
353         /**
354          * @brief Devuelve el nombre del donante que ha donado la mayor cantidad de dinero
355          * desde que se registró en el sistema. Si hay varios donantes con la misma cantidad,
356          * se devuelve el que primero se encuentre en la lista de donantes registrados.
357          * @return String. El nombre del donante.
358          * @throws RemoteException Si ocurre un error de comunicación remota.
359          */
360         @Override
361         public String mayorDonanteGlobal() throws RemoteException {
362                 // 1. Obtenemos TODOS los donantes del sistema
363                 // Primero obtenemos los donantes registrados en esta réplica.
364                 List<String> todosLosDonantes = this.listarDonantesLocal();
365                 for(iDonacion peer : peers.values()) {
366                         // Ahora obtenemos los donantes registrados en las otras réplicas.
367                         todosLosDonantes.addAll(peer.listarDonantesLocal());
368                 }
369
370                 String mayorDonante = null;
371                 int maxCantidad = -1;
```

```
372
373                 // 2. Iteramos para encontrar el donante
374                 for(String d : todosLosDonantes) {
375                         int totalDeEsteDonante = this.getTotalDonacionesDonante(d); // Consulta su total
376
377                         // Si la cantidad es estrictamente MAYOR, se actualiza el máximo.
378                         // Si es IGUAL, no se actualiza, por lo que Conservamos el que aparece antes en la lista.
379                         if(totalDeEsteDonante > maxCantidad) {
380                                 maxCantidad = totalDeEsteDonante;
381                                 mayorDonante = d;
382                         }
383                 }
384
385                 return mayorDonante;
386         }
387
388         /**
389      * @brief Permite a un donante consultar su historial de donaciones.
390      * @pre El donante debe estar registrado previamente y haber realizado al menos una donación.
391      * @param donante El nombre del donante que quiere consultar su historial.
392      * @return List<Integer>. Lista con las cantidades donadas.
393      * @throws RemoteException Si ocurre un error de comunicación remota.
394      */
395         @Override
396     public List<Integer> obtenerHistorialDonaciones(String donante) throws RemoteException {
397                 List<Integer> historial = null;
398                 boolean encontrado = false;
399
400                 // 1. Comprobamos de forma segura si el donante está en esta réplica
401                 synchronized(this) {
402                         encontrado = this.estaRegistrado(donante);
403                         if (encontrado) {
404                                 // Comprobamos la precondición de que haya realizado al menos una donación
405                                 if (!donantes.get(donante).isEmpty()) {
406                                         // Devolvemos una copia por seguridad
407                                         historial = new ArrayList<>(donantes.get(donante));
408                                 } else {
409                                         System.out.println("[ERROR.] El donante "
410                                         + donante + " no ha realizado ninguna donación.");
411                                 }
412                         }
413                 }
414
415                 if(!encontrado) {
416                         // 2. Si no está en esta réplica local, buscamos en qué réplica (peer)
417                         // está registrado
418                         for (iDonacion peer : peers.values()) {
419                                 if (peer.estaRegistrado(donante)) {
420                                         // Si lo encontramos en un peer, le pedimos la información
421                                         // y cortamos la búsqueda
422                                         historial = peer.obtenerHistorialDonaciones(donante);
423                                         encontrado = true;
424                                         break;
425                                 }
426                         }
427                 }
428
429                 // Si llegados este punto no lo ha encontrado en ninguna réplica
430                 if(!encontrado) {
431                         System.out.println("[ERROR.] El donante "
432                         + donante + " no está registrado en el sistema.");
433                 }
```

```
434
435                 return historial;
436         }
437 }
```

3.2.3. `Servidor.java` (El Servidor)

Es el servidor encargado de publicar el objeto remoto `Donacion`. Implementa la seguridad `SSL/TLS`, haciendo uso de las clases `SslRMIClientSocketFactory` y `SslRMIServerSocketFactory` para cifrar la comunicación entre el cliente y el servidor. Además, **se ha implementado de forma que se puedan crear tantas réplicas como se quiera, simplemente lanzando el servidor varias veces con un nombre de nodo distinto cada vez**, y estas réplicas se comunicarán entre sí para compartir información sobre los donantes registrados y las donaciones realizadas. Al arrancar el servidor, **hace un descubrimiento de las otras réplicas (peers) del sistema**, para ello, se conecta al registro (el `rmiregistry`) y busca los objetos remotos de las otras réplicas por su nombre, y si los encuentra, añade una referencia a estos objetos remotos en su lista de pares. Esto permite que las réplicas puedan comunicarse entre sí para compartir información sobre los donantes registrados y las donaciones realizadas. Además, el servidor se encarga de exportar el objeto remoto `Donacion` con SSL, para ello, se crea una instancia de este objeto remoto y se exporta. Después se registra en el registro con su nombre, para que los clientes puedan encontrarlo y conectarse a él. El código de este componente es:

```
1  /**
2   * @file Servidor.java
3   * @brief Código del servidor que implementa la lógica de negocio y publica el objeto remoto en
4   * el RMI Registry. Este servidor es replicado, lo que significa que pueden lanzarse varias
5   * instancias de este servidor, cada una con un nombre único, y se conocen entre ellas para
6   * replicarse mutuamente.
7   */
8  import java.rmi.RemoteException;
9  import java.rmi.registry.LocateRegistry;
10 import java.rmi.registry.Registry;
11 import java.rmi.server.ExportException;
12 import java.rmi.server.UnicastRemoteObject;
13 import javax.rmi.ssl.SslRMIClientSocketFactory;
14 import javax.rmi.ssl.SslRMIServerSocketFactory;
15
16 public class Servidor {
17
18     private static final int RMI_PORT = 2099;
19
20     /**
21      * @brief Genera un nombre único para cada servidor que se lance, basado en el
22      * número de servidores que ya están registrados en el RMI Registry. Así, los
23      * servidores se llamarán Donacion_1, Donacion_2, etc.
24      * @param registry El RMI Registry donde se van a registrar los servidores.
```

```
25          * @return String. El nombre único generado para el nuevo servidor.
26          */
27             private static String generateName(Registry registry) throws RemoteException {
28                     int n = 0;
29             // Con registry.list() obtenemos los nombres de los objetos
30             // registrados en el RMI Registry y miramos cuántos de ellos
31             // son de tipo "Donacion_" para asignar un nombre incremental
32             // a cada nuevo objeto que se lance. Así es como se llamarán
33             // nuestros servidores: Donacion_1, Donacion_2, ...
34                     for(String name : registry.list()){
35                             if(name.contains("Donacion_")){
36                                     n++;
37                             }
38                     }
39                     return "Donacion_" + (n+1);
40             }
41
42         /**
43          * @brief Método principal del servidor. Crea el RMI Registry si no existe,
44          * genera un nombre único para el servidor, crea una instancia del objeto
45          * remoto Donacion, la exporta y la registra en el RMI Registry. Luego,
46          * busca otros servidores registrados para conocerse entre ellos y replicarse.
47          * @param args Argumentos de línea de comandos (no se utilizan en este caso).
48          */
49         public static void main(String[] args) {
50
51             // Creamos las fábricas de sockets SSL para el cliente y el servidor
52             SslRMIClientSocketFactory csf = new SslRMIClientSocketFactory();
53             SslRMIServerSocketFactory ssf = new SslRMIServerSocketFactory();
54
55             // Creamos el registry en este mismo proceso si no existe.
56             try {
57                 LocateRegistry.createRegistry(RMI_PORT, csf, ssf);
58                 System.out.println("RMI Registry creado en el puerto " + RMI_PORT);
59             } catch (ExportException e) {
60                 System.out.println("Un RMI Registry ya estaba activo en el puerto "
61                                 + RMI_PORT);
62             } catch (RemoteException e) {
63                 System.out.println("No se pudo iniciar el RMI Registry en el puerto "
64                                 + RMI_PORT + ": " + e.getMessage());
65                 return;
66             }
67
68             try {
69                 // Obtenemos el registro (si el registry va con SSL, usa csf aquí también)
70                 Registry registry = LocateRegistry.getRegistry("localhost", RMI_PORT, csf);
71
72                 // Determinar dinámicamente el nombre incremental
73                 String ob_name = generateName(registry);
74
75                 // Creamos el objeto
76                 Donacion gestor = new Donacion(ob_name);
77
78                 // Generamos un stub y lo publicamos
79                 iDonacion stub0 = (iDonacion) UnicastRemoteObject.exportObject(gestor, 0, csf, ssf);
80                 registry.rebind(ob_name, stub0);
81
82                             // registry.lookup() devuelve un java.rmi.Remote genérico, hay que castearlo explícitamente a
83                             iDonacion obLocal = (iDonacion) registry.lookup(ob_name);
84                             for(String name : registry.list()){
85                                     if(name.contains("Donacion_") && !name.equals(ob_name)){
86                                             iDonacion obRemoto = (iDonacion) registry.lookup(name);
```

```
87                                          // setPeer sirve para que cada servidor conozca a los demás servidores
88                      // y puedan replicarse entre ellos.
89                                          obLocal.setPeer(obRemoto);
90                                          obRemoto.setPeer(obLocal);
91                                      }
92                                  }
93
94              System.out.println("Servidor preparado. Lanzado el objeto: " + ob_name);
95
96          } catch (RemoteException e) {
97              System.out.println("No se pudieron registrar los objetos remotos: " + e.getMessage());
98          } catch (java.rmi.NotBoundException e) {
99              System.out.println("No se encontró el objeto remoto en el registro: " + e.getMessage());
100         }
101     }
102 }
```

3.2.4. `Cliente.java` (El Cliente)

Es el cliente encargado de interactuar con el objeto remoto `Donacion`. Tiene como atributos el *host* del servidor y el puerto donde se encuentra el objeto remoto, además del nombre del primer servidor, que es Donacion_1. Después, realiza las siguientes acciones:

- Crea la fábrica de sockets SSL para el cliente, para cifrar la comunicación con el servidor.
- Obtiene el registro del servidor y busca el objeto remoto de la réplica del servidor con el que se va a conectar, que es la réplica donde se ha registrado el cliente. Para ello, se conecta al registro y busca el objeto remoto por su nombre, que es "Donacion_i", donde i es un entero entre 1 y el número de réplicas que hay en total. Si encuentra el objeto remoto, lo almacena en un atributo para poder interactuar con él posteriormente.
- A continuación entra en un bucle donde muestra un menú de opciones al cliente, para que este pueda elegir la acción que desea realizar. Las opciones son: registrar un nuevo donante, dar de baja a un donante registrado, realizar una donación, consultar el total donado por un donante, consultar el listado de donantes registrados en el sistema, consultar el total de donaciones realizadas por el donante que la llama, consultar el nombre del mayor donante global, consultar el historial de donaciones del donante que lo llama y salir del programa. En función de la opción elegida por el cliente, se llama al método correspondiente del objeto remoto para realizar la acción deseada. Por ejemplo, si el cliente elige la opción de registrar un nuevo donante, se le pide que

introduzca su nombre, y se llama al método `registrar` del objeto remoto, pasándole el nombre del donante como parámetro. Si el cliente elige la opción de realizar una donación, se le pide que la cantidad a donar, y se llama al método `donar` del objeto remoto (aunque previamente debe estar registrado, si no, se le pide que se registre primero). Y así sucesivamente con el resto de opciones.

- Otro aspecto a destacar es que **tanto en el cliente como en el resto de componentes se tiene en cuenta la gestión de excepciones**, ya que se han añadido bloques `try-catch` para manejar las posibles excepciones que puedan ocurrir durante la ejecución del programa, como por ejemplo, problemas de conexión con el servidor, errores al buscar el objeto remoto en el registro, errores al llamar a los métodos remotos, etc. Esto permite que el programa sea más robusto y pueda manejar situaciones inesperadas de forma adecuada.

El código del cliente es:

```java
/**
 * @file Cliente.java
 * @brief Cliente RMI para el sistema de donaciones replicado.
 */

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.List;
import java.util.Scanner;
import javax.rmi.ssl.SslRMIClientSocketFactory;

public class Cliente {

    static void mostrarSeparador(Scanner scanner) {
        System.out.println("\n ----- Presione Enter para continuar -----");
        scanner.nextLine(); // Espera a que el usuario pulse Enter
        for (int i = 0; i < 30; i++) {
            System.out.println(); // Añade muchos saltos de línea para limpiar la pantalla
        }
    }

    public static void main(String[] args) {

        // Host y puerto donde se encuentra el RMI Registry
        String host = "localhost";
        int puerto = 2099;

        // Nombre del servidor al que nos conectaremos inicialmente (punto de entrada)
        // Asumimos que al menos existe una réplica llamada "Donacion_1"
        String nombreServidor = "Donacion_1";

        try {
            // Como el servidor usa SSL, el cliente también debe usar la fábrica SSL
            SslRMIClientSocketFactory csf = new SslRMIClientSocketFactory();

            // Obtenemos el RMI Registry
```

```
37              Registry registry = LocateRegistry.getRegistry(host, puerto, csf);
38
39              // Buscamos el objeto remoto (servidor) inicial
40              iDonacion servidor = (iDonacion) registry.lookup(nombreServidor);
41              System.out.println("Se ha establecido la conexión con el servidor inicial: " + servidor.getNombre());
42
43              Scanner scanner = new Scanner(System.in);
44              int opcion = 0; // Opción seleccionada del menú
45              String miNombre = ""; // Guardará el nombre del donante activo en esta sesión
46
47              while (opcion != 9) { // El menú se repetirá hasta que el usuario elija salir (opción 9)
48                  System.out.println("\n\n\n------- MENÚ DE CLIENTE (RMI) --------------------------------");
49                  System.out.println("Nombre de usuario: "
50                                     + (miNombre != "" ? miNombre : "Ninguno"));
51                  System.out.println("Réplica actual asignada: " + servidor.getNombre());
52                  System.out.println("------------------------------");
53                  System.out.println("1. Registrarme como donante / Iniciar sesión (si ya estoy registrado)");
54                  System.out.println("2. Darme de baja como donante");
55                  System.out.println("3. Realizar donación");
56                  System.out.println("4. Consultar total de donaciones (de forma global)");
57                  System.out.println("5. Listar todos los donantes (de forma global)");
58                  System.out.println("6. Consultar el mayor donante registrado en el sistema");
59                  System.out.println("7. Consultar el total de mis donaciones");
60                  System.out.println("8. Consultar mi historial de donaciones");
61                  System.out.println("9. Salir");
62                  System.out.println("-------------------------------------------------------------------------");
63                  System.out.print("Elija una opción: ");
64
65                  try {
66                      opcion = Integer.parseInt(scanner.nextLine());
67                  } catch (NumberFormatException e) {
68                      System.out.println("Opción inválida. Introduzca un número (del 1 al 9).");
69                      continue;
70                  }
71
72                  switch (opcion) {
73                      case 1: // Registrar donante
74                          System.out.print("Introduzca el nombre del donante a registrar: ");
75                          miNombre = scanner.nextLine();
76                          // Al llamar a registrar, el servidor puede devolvernos la referencia
77                          // de otra réplica (balanceo de carga). Es pro eso que debemos actualizar
78                          // nuestra referencia.
79                          servidor = servidor.registrar(miNombre);
80                          System.out.println("[ÉXITO] Registrado correctamente. Asignado a la réplica: "
81                                             + servidor.getNombre());
82                          break;
83                      case 2: // Dar de baja a un donante
84                          if (miNombre.isEmpty()) {
85                              System.out.println("[ERROR] Primero debe registrarse (Opción 1).");
86                          } else {
87                              servidor.darDeBaja(miNombre);
88                              System.out.println("[ÉXITO] Dado de baja correctamente.");
89                              miNombre = "";
90                          }
91                          break;
92
93                      case 3: // Realizar donación
94                          if (miNombre.isEmpty()) {
95                              System.out.println("[ERROR] Primero debe registrarse (Opción 1).");
96                          } else {
97                              System.out.print("Introduzca la cantidad a donar (en €): ");
98                              try {
```

```
                    int cantidad = Integer.parseInt(scanner.nextLine());
                    if(cantidad <= 0) {
                        System.out.println("[ERROR] La cantidad debe ser mayor que 0.");
                    } else {
                        // Si ha introducido una cantidad válida, realizamos la donación
                        servidor.donar(miNombre, cantidad);
                        System.out.println("[ÉXITO] Donación de " + cantidad
                                           + "€ realizada correctamente.");
                    }
                } catch (NumberFormatException e) {
                    System.out.println("[ERROR] La cantidad introducida no es válida.");
                }
            }
            break;

        case 4: // Consultar total de donaciones (global)
            if (miNombre.isEmpty()) {
                System.out.println("[ERROR] Primero debe registrarse (Opción 1).");
            } else {
                int total = servidor.totalDonaciones(miNombre);
                if (total != -1) {
                    System.out.println("[RESULTADO] El total donado en todo el sistema es: "
                                       + total + "€");
                } else { // Si el resultado es -1, significa que no se cumplen las precondiciones para co
                    System.out.println("[DENEGADO] No puede consultar. Compruebe que ha donado más de 0€
                }
            }
            break;

        case 5: // Listar todos los donantes (global)
            if (miNombre.isEmpty()) {
                System.out.println("[ERROR] Primero debe registrarse (Opción 1).");
            } else {
                List<String> listado = servidor.listarDonantes(miNombre);
                if (listado != null && !listado.isEmpty()) {
                    System.out.println("[RESULTADO] Lista global de donantes registrados:");
                    for (String d : listado) {
                        System.out.println(" - " + d);
                    }
                } else {
                    System.out.println("[DENEGADO] No puede listar donantes. Compruebe que ha donado más
                }
            }
            break;

        case 6: // Consultar el mayor donante registrado en el sistema
            System.out.println("[RESULTADO] El mayor donante registrado en el sistema es: "
                               + servidor.mayorDonanteGlobal() + " (cantidad donada: "
                               + servidor.getTotalDonacionesDonante(servidor.mayorDonanteGlobal()) + "€)
            break;

        case 7: // Consultar el total de mis donaciones
            if (miNombre.isEmpty()) {
                System.out.println("[ERROR] Primero debe registrarse (Opción 1).");
            } else {
                int totalMisDonaciones = servidor.getTotalDonacionesDonante(miNombre);
                System.out.println("[RESULTADO] El total de sus donaciones es: " + totalMisDonaciones + "
            }
            break;

        case 8: // Consultar mi historial de donaciones
            if (miNombre.isEmpty()) {
```

```
                            System.out.println("[ERROR] Primero debe registrarse (Opción 1).");
                        } else {
                            List<Integer> historial = servidor.obtenerHistorialDonaciones(miNombre);
                            if (historial != null && !historial.isEmpty()) {
                                System.out.println("[RESULTADO] Su historial de donaciones:");
                                for (int i = 0; i < historial.size(); i++) {
                                    System.out.println(" Donación " + (i+1) + ": " + historial.get(i) + "€");
                                }
                            } else {
                                System.out.println("[RESULTADO] No ha realizado ninguna donación aún.");
                            }
                        }
                        break;

                    case 9: // Salir
                        System.out.println("Desconectando y cerrando el cliente...");
                        break;

                    default:
                        System.out.println("La opción introducida no es válida. Por favor, seleccione del 1 al 9.");
                }

                if(opcion != 9)
                    mostrarSeparador(scanner);
            }
            scanner.close();

        } catch (Exception e) {
            System.err.println("Excepción en el cliente:");
            e.printStackTrace();
        }
    }
}
```

3.2.5. `launch.sh` (Script para Ejecución)

Se ha modificado el script que nos proporcionaban en los ejemplos. Se compilan los archivos de la parte 2, y se lanzan las dos réplicas del servidor y el cliente, dándose un delay para que se inicialicen correctamente y se emparejen.

```
#!/bin/bash

# Configuración de variables
KEYSTORE=server-keystore.p12
TRUSTSTORE=client-truststore.p12
STOREPASS=changeit
RMI_PORT=2099

echo "================================================"
echo "   Iniciando Sistema de Donaciones (RMI SSL)     "
echo "================================================"

# 1. Generación de certificados SSL (Si no existen)
if [ ! -f "$KEYSTORE" ]; then
    echo "[1/5] Generando keystore TLS para el servidor..."
    keytool -genkeypair \
```

```
        -alias rmi-server \
        -keyalg RSA \
        -keysize 2048 \
        -validity 365 \
        -dname "CN=localhost, OU=DSD, O=UGR, L=Granada, ST=Granada, C=ES" \
        -storetype PKCS12 \
        -keystore "$KEYSTORE" \
        -storepass "$STOREPASS" \
        -keypass "$STOREPASS" \
        -noprompt
fi

if [ ! -f "$TRUSTSTORE" ]; then
    echo "[2/5] Generando truststore TLS para el cliente..."
    keytool -exportcert \
        -alias rmi-server \
        -keystore "$KEYSTORE" \
        -storepass "$STOREPASS" \
        -file rmi-server.crt \
        -rfc
    keytool -importcert \
        -alias rmi-server \
        -file rmi-server.crt \
        -keystore "$TRUSTSTORE" \
        -storepass "$STOREPASS" \
        -noprompt
    rm rmi-server.crt
fi

# 2. Limpieza de procesos (Matar cualquier servidor previo en el puerto 2099)
echo "[3/5] Limpiando procesos previos en el puerto $RMI_PORT..."
fuser -k $RMI_PORT/tcp > /dev/null 2>&1

# 3. Compilación de todos los archivos
echo "[4/5] Compilando los archivos Java..."
javac *.java

if [ $? -ne 0 ]; then
    echo "Error en la compilación. Abortando ..."
    exit 1
fi

# 4. Lanzamiento de las Réplicas
echo "[5/5] Levantando las réplicas del servidor..."

# Lanzar Réplica 1 (Esta creará el Registry y se llamará Donacion_1)
java -cp . \
    -Djava.rmi.server.hostname=localhost \
    -Djavax.net.ssl.keyStore="$KEYSTORE" \
    -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
    -Djavax.net.ssl.keyStoreType=PKCS12 \
    -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
    -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
    -Djavax.net.ssl.trustStoreType=PKCS12 \
    Servidor &

# Le damos 2 segundos para que arranque bien antes de lanzar la siguiente
sleep 2

# Lanzar Réplica 2 (Esta detectará a la 1 y se llamará Donacion_2)
java -cp . \
    -Djava.rmi.server.hostname=localhost \
```

```
    -Djavax.net.ssl.keyStore="$KEYSTORE" \
    -Djavax.net.ssl.keyStorePassword="$STOREPASS" \
    -Djavax.net.ssl.keyStoreType=PKCS12 \
    -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
    -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
    -Djavax.net.ssl.trustStoreType=PKCS12 \
    Servidor &

# Le damos otros 2 segundos para que se conecten entre ellas
sleep 2

echo "================================================"
echo "   Servidores listos. Lanzando el Cliente...     "
echo "================================================"

# 5. Lanzamiento del Cliente
java -cp . \
    -Djavax.net.ssl.trustStore="$TRUSTSTORE" \
    -Djavax.net.ssl.trustStorePassword="$STOREPASS" \
    -Djavax.net.ssl.trustStoreType=PKCS12 \
    Cliente localhost
```

3.3. Diagramas en UML.

3.3.1. Diagrama de clases.

El diagrama de clases del sistema que hemos descrito es el siguiente:

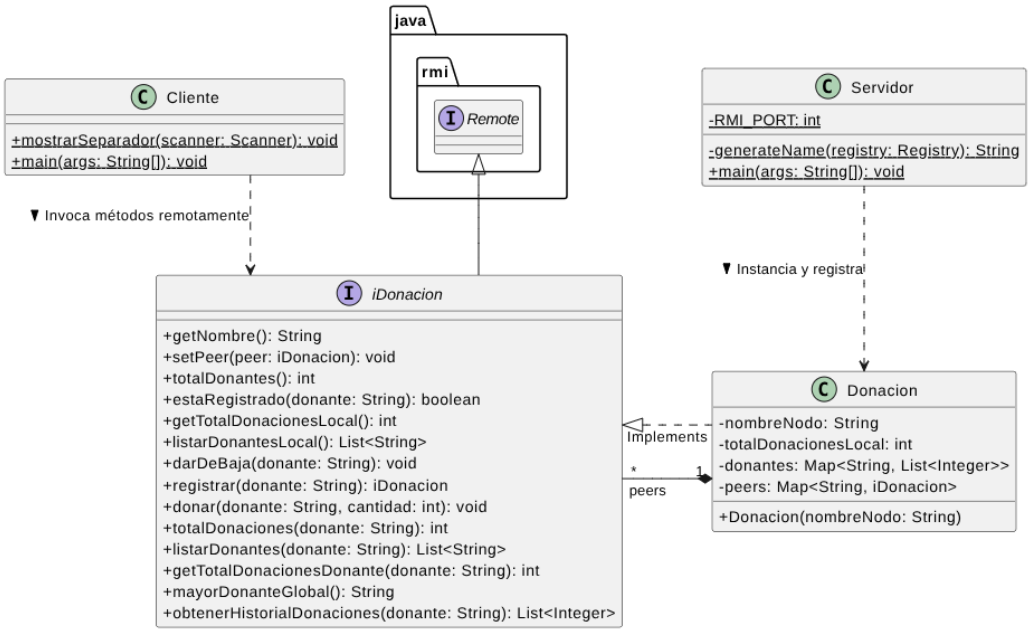


Figura 9: Diagrama de clases de la parte 2 Servidor Replicado

3.3.2. Diagrama de clases.

El diagrama de secuencia muestra un ejemplo de varias operaciones del sistema como: registrar un donante, realizar una donación y pedir el total de donaciones.

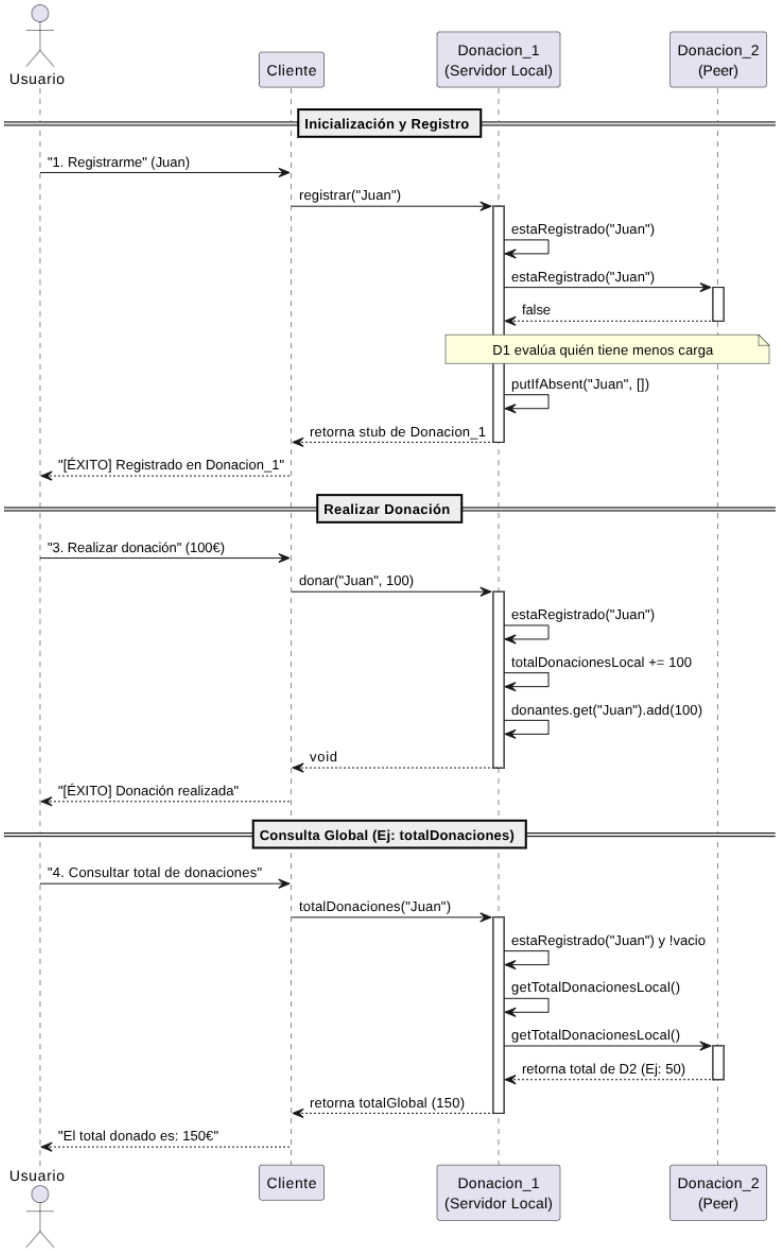


Figura 10: Diagrama de secuencia de la parte 2 Servidor Replicado

3.4. Guía de Uso. Ejemplos de ejecución

En este apartado vamos a ver como compilar y ejecutar nuestra aplicación y vamos a realizar un par de operaciones de ejemplo.

Lo primero que tendremos que hacer es dar permisos de ejecución a nuestro script. Para ello abrimos una terminal y ejecutamos el siguiente comando:

```
$ chmod u+x launch.sh
```

Después, ejecutamos el script con el siguiente comando:

```
$ ./launch.sh
```

Llegados a este punto, hemos hecho: generado las keystore TLS para el servidor y las del cliente, limpiado los procesos previos que habia en el puerto que usamos (que es el 2099), compilado los archivos `.java` necesarios y, lo más importante, **se lanzan dos réplicas del servidor en segundo plano, y un cliente**. La salida es la que se muestra en la siguiente imagen:

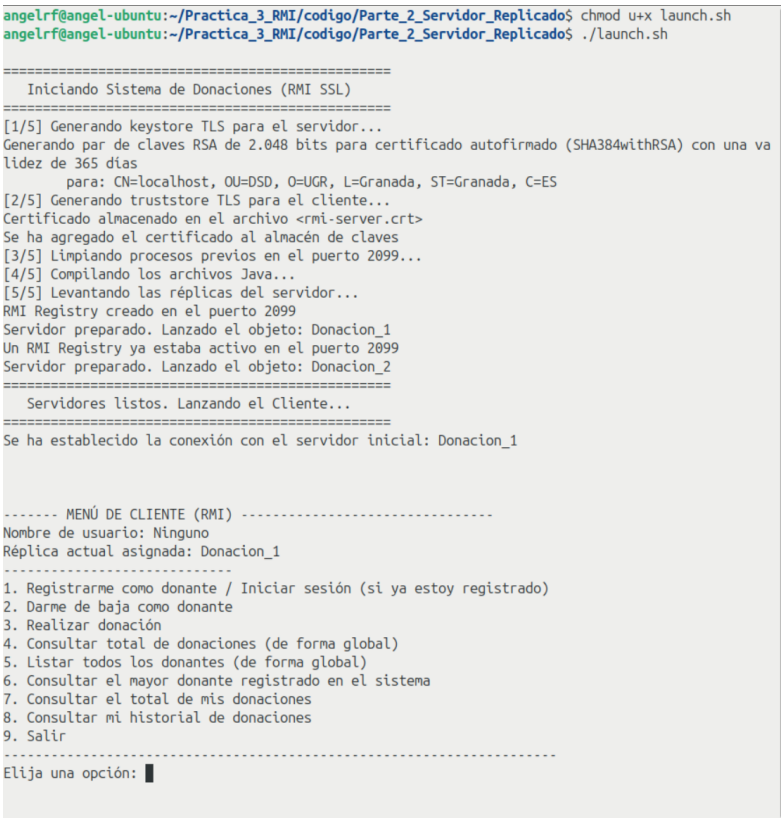


Figura 11: Salida al lanzar el script de la parte 2 Servidor Replicado

Ahora, abrimos una nueva terminal y **lanzamos un segundo cliente** con el comando:

$ java -cp . -Djavax.net.ssl.trustStore=çlient-truststore.p12-Djavax.net.ssl.trustStorePassword=çhangeit-Djavax.net.ssl.trustStoreType=PKCS12 Cliente

La salida que veamos será igual que la de la siguiente imagen (figura 12):

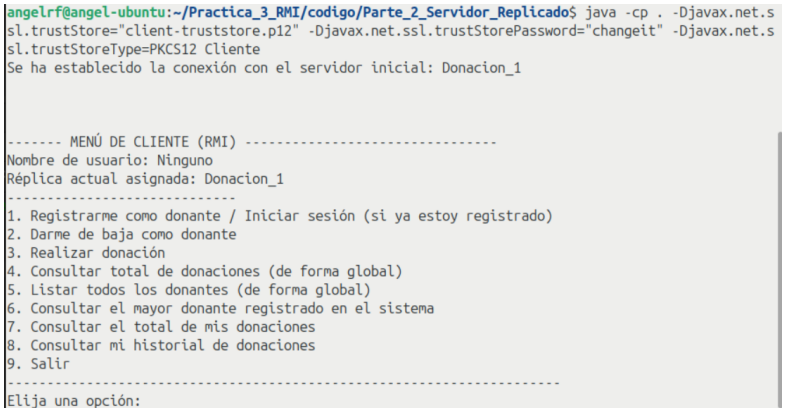


Figura 12: Salida al lanzar el segundo cliente de la parte 2 Servidor Replicado

Ahora, vamos a ir realizando varias operaciones de ejemplo:

3.4.1. Registrar donantes.

Primero vamos a registrar un donante, haciendo uso del Cliente 1. El donante se llamará "Pablo". (Figura 13)



Figura 13: Registrar el 1er donante

Como podemos observar, se ha registrado con éxito, y nos indica que se ha registrado en la réplica `Donacion_1`

Vamos a registrar un segundo donante, ahora en el Cliente 2. La donante se llamará .ªlba"(Figura 14)



Figura 14: Registrar el 2º donante

Como podemos observar, también se ha registrado con éxito, y nos indica que se ha registrado en la réplica `Donacion_2`. **Recordemos el balanceo de carga**

3.4.2. Realizar donaciones

Ahora vemos que arriba en el menú ya nos aparece el nombre del usuario y la replica donde se encuentra, **si volvemos a seleccionar la opción 1 y registramos un nuevo cliente,se cambiará y estaremos en el nuevo cliente. En cambio, si introducimos el nombre de un usuario que ya existe, se nos cambiará a ese usuario (como si iniciáramos sesión con él)**.

Mediante el Cliente 1, Pablo va enviar una donación de 10€ (Figura 15)

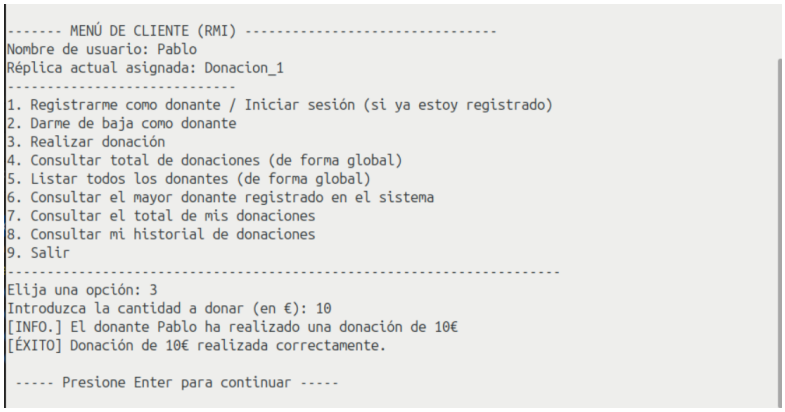


Figura 15: Donación de Pablo de 10€ a través del cliente 1

Ahora Alba, mediante el Cliente 2, va enviar dos donaciones, una de 15€ (Figura 16) y otra de 8€ (Figura 17)

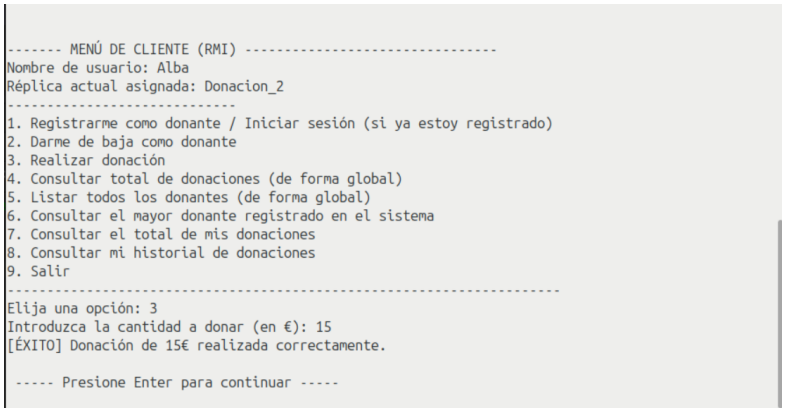


Figura 16: Donación de Alba de 15€ a través del cliente 2

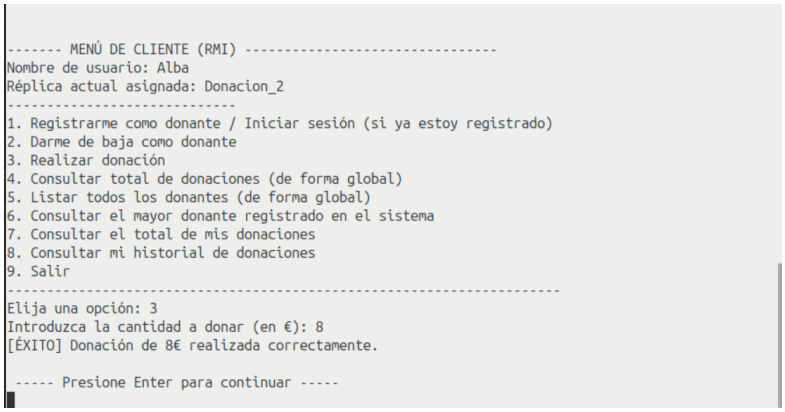


Figura 17: Donación de Alba de 8€ a través del cliente 2

3.4.3. Consultar total de donaciones (en todo el sistema)

Pablo, a través del cliente 1, consulta la cantidad total de dinero que se ha donado:

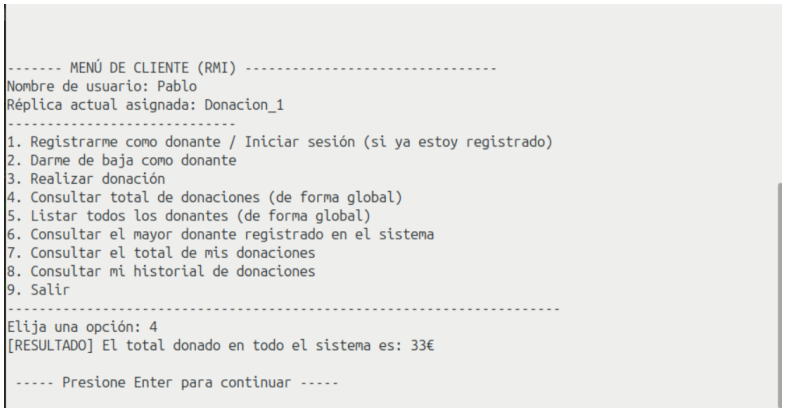


Figura 18: Consulta de Pablo de total de donaciones a través del cliente 1

Vemos que es la suma de las tres donaciones realizadas (10 + 15 + 8)€ = 33€.

Nota: En la terminal 1 donde está el cliente 1, **también se muestran los mensajes del servidor, ya que se ejecutan en segundo plano**, un ejemplo de ello es la figura 19:

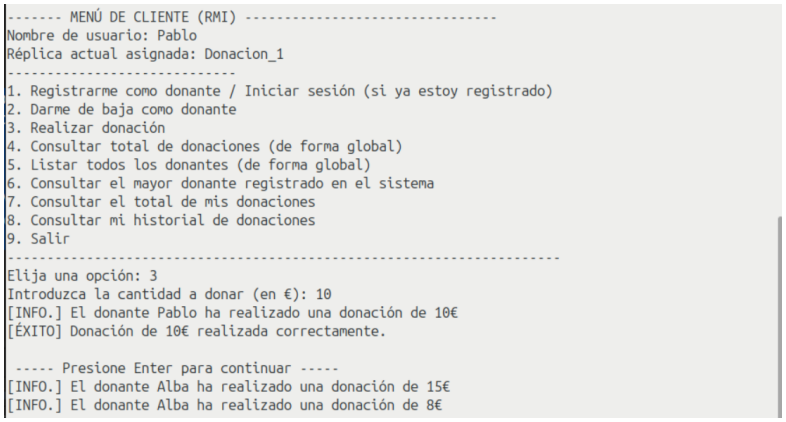


Figura 19: Ejemplo de mensajes del servidor a través del cliente 1

3.4.4. Consultar el historial de donaciones

Alba, a través del cliente 2, consulta el historial de donaciones (opcion = 8) que ha realizado desde que se dio de alta en el sistema: (Figura 20)

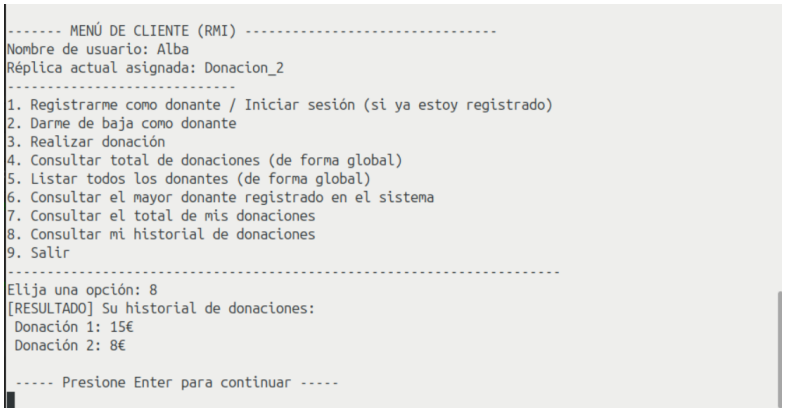


Figura 20: Consulta de Alba de historial de donaciones a través del cliente 1, al final del todo

3.4.5. Consultar el mayor donante.

Pablo, a través del cliente 1, consulta el mayor donante del que se tiene registro (opcion = 6) que ha realizado desde que se dio de alta en el sistema: (Figura 21)

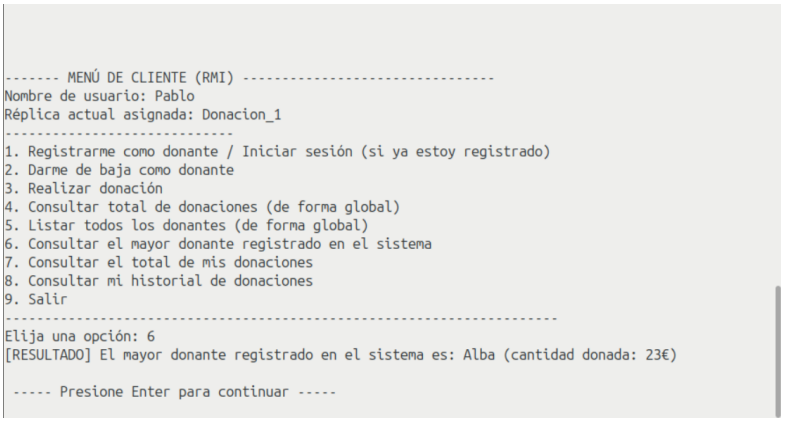


Figura 21: Consulta del mayor donante del sistema

3.4.6. Darse de baja.

Pablo, a través del cliente 1, se da de baja del sistema: (Figura 22)

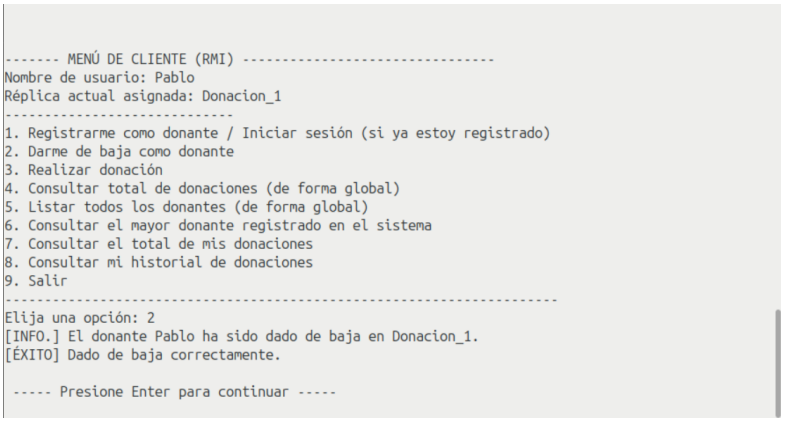


Figura 22: Ejemplo de darse de baja

3.4.7. Salir de la aplicación.

Alba , a través del cliente 2, sale de la aplicación



Figura 23: Ejemplo de salir

Estos son algunos ejemplos de operaciones que se pueden realizar. Hay muchas más combinaciones o posibilidades por probar (como meter valores mal a propósito para demostrar que el sistema es robusto ante fallos y demás), pero no quiero extender más este documento.